.NET Core & C# Interview Guide

C#, OOP/SOLID, LINQ, Async, ASP.NET Core, EF Core, SQL, Architecture

Theory + practical + real-world scenario questions, with solutions

Interview Prep Series | Prepared for Mustafa

Contents

1. C# Language Fundamentals 3
2. OOP and SOLID 7
3. Collections and LINQ 8
4. Async, Multithreading and Concurrency 10
5. ASP.NET Core Fundamentals 12
6. Security 16
7. Entity Framework Core and Data Access 18
8. Design Patterns and Architecture 20
9. Testing 22
10. Coding Questions in C# 23
11. Real-World Scenario Questions 25
12. Quick-Fire Questions 27
13. OOP Deep Dive: Every Concept and Tricky Questions 29
14. C# Tricky Output Questions 36
15. More C# Language Questions 41
16. Collections and LINQ (More) 46
17. Async, Threading and Concurrency (More) 49
18. ASP.NET Core (More In-Depth) 52
19. Entity Framework Core (More) 57
20. SQL Server Questions for .NET Developers 59
21. Architecture, Microservices and DevOps (More) 62
22. Scenario and Rapid-Fire Round 64

1. C# Language Fundamentals

Q1. Value types vs reference types.

Value types (`int`, `decimal`, `bool`, `struct`, `enum`, `DateTime`) hold their data directly; assignment copies the value. Reference types (`class`, `string`, arrays, `delegate`, `record class`) hold a reference to an object on the managed heap; assignment copies the reference. Value types are not "always on the stack": a value-type field inside a class lives on the heap with the object.

```
struct P { public int X; }      class C { public int X; }
var a = new P { X = 1 }; var b = a; b.X = 2;   // a.X = 1
var c = new C { X = 1 }; var d = c; d.X = 2;   // c.X = 2
```

Q2. Stack vs heap.

The stack stores method frames: locals, parameters and return addresses; it's freed automatically when the method returns. The heap stores objects whose lifetime is managed by the Garbage Collector. Large objects (85,000 bytes or more) go to the Large Object Heap, which is collected with Gen 2.

Q3. What is boxing and unboxing? Why avoid it?

Boxing wraps a value type in an object on the heap; unboxing extracts it back with a cast. It costs allocations and GC pressure. Common hidden boxing: non-generic `ArrayList`, `object` parameters, string interpolation of structs in older compilers, interfaces on structs.

```
int i = 10; object o = i;   // boxing
int j = (int)o;             // unboxing (InvalidCastException if wrong type)
```

Q4. string vs StringBuilder. Why are strings immutable?

`string` is immutable: every concatenation creates a new string. In a loop that becomes O(n squared) work and lots of garbage. `StringBuilder` uses a resizable buffer. Immutability gives thread-safety, safe use as dictionary keys and string interning.

```
var sb = new StringBuilder();
foreach (var t in txns) sb.Append(t.Rrn).Append('|').AppendLine(t.Amount.ToString("F2"));
```

Q5. ref vs out vs in.

- `ref`: argument must be initialised; method may read and change it.
- `out`: need not be initialised; method must assign it (`int.TryParse(s, out var n)`).
- `in`: passed by reference but read-only; avoids copying large structs.

Q6. const vs readonly vs static readonly.

`const` is compile-time, implicitly static, and its value is baked into calling assemblies (changing it requires recompiling consumers). `readonly` is set at declaration or in the constructor, per instance. `static readonly` is set once per type at runtime; use it for values like `TimeSpan` or config objects.

Q7. What are access modifiers?

`public`, `private` (default for members), `protected` (class + derived), `internal` (same assembly, default for top-level types), `protected internal` (assembly OR derived), `private protected` (derived types in the same assembly).

Q8. Abstract class vs interface.

Abstract class: can have state (fields), constructors and implemented members; single inheritance; models "is-a" with shared base behaviour. Interface: a contract; a class can implement many; since C# 8 can have default implementations and static members, but no instance fields. Choose interface for capabilities and dependency injection (`IPaymentGateway`), abstract class when sharing code across closely related types (`ReconRuleBase`).

Q9. Method overloading vs overriding; new vs override.

Overloading: same name, different parameters (compile-time polymorphism). Overriding: derived class replaces a `virtual`/`abstract` base method (run-time polymorphism). `new` hides the base method: the method called depends on the **declared** type of the variable, not the actual object.

```
class A { public virtual string M() => "A"; public string N() => "A"; }
class B : A { public override string M() => "B"; public new string N() => "B"; }
A x = new B(); x.M(); // "B"   x.N(); // "A"
```

Q10. What is a sealed class?

It cannot be inherited (`sealed class`); a sealed override stops further overriding. Used for security, design intent, and small JIT performance gains (devirtualisation). `string` is sealed.

Q11. What is a static class and a static constructor?

A static class cannot be instantiated or inherited and contains only static members (helpers, extension methods). A static constructor runs once, automatically, before the first use of the type; thread-safe by the runtime.

Q12. What are extension methods?

Static methods in a static class with `this` on the first parameter, which appear as instance methods. LINQ is built on them.

```
public static class DecimalExt {
    public static decimal ToPaise(this decimal rupees) => rupees * 100m;
}
var p = 12.5m.ToPaise();
```

Q13. class vs struct vs record.

Struct: value type, copied on assignment, best for small immutable data (under ~16 bytes) like `Money`, `Point`. Class: reference type, identity-based equality. Record (C# 9): reference type with **value-based equality**, `with` expressions, concise positional syntax; ideal for DTOs and events. `record struct` (C# 10) is the value-type version.

```
public record TxnDto(string Rrn, decimal Amount);
var a = new TxnDto("1", 10); var b = a with { Amount = 20 };
new TxnDto("1", 10) == new TxnDto("1", 10)   // true
```

Q14. What are nullable reference types?

C# 8 feature that makes the compiler warn when a possibly-null reference is dereferenced. `string?` may be null; `string` should not be. Operators: `?.`, `??`, `??=`, and `!` (null-forgiving, use sparingly).

Q15. Equals vs == vs ReferenceEquals; GetHashCode rule.

`==` for reference types compares references unless overloaded (string overloads it for value comparison). `Equals` is virtual and can be overridden. `ReferenceEquals` always compares references. Rule: if you override `Equals`, override `GetHashCode` so equal objects have equal hash codes, otherwise Dictionary/HashSet break.

Q16. Why use decimal instead of double for money?

`double` is binary floating point and cannot exactly represent values like 0.1 (`0.1 + 0.2 != 0.3`). `decimal` is base-10 with 28-29 significant digits; exact for currency. Use `decimal` in C# and `DECIMAL(18,2)` (or more scale) in SQL Server, and be explicit with rounding (`MidpointRounding.AwayFromZero` vs banker's rounding, the default).

Q17. What is pattern matching in modern C#?

```
string Classify(Txn t) => t switch {
    { Amount: > 100000 } => "HighValue",
    { Status: "REVERSED" } => "Reversal",
    { Channel: "POS", Amount: < 0 } => "Refund",
    _ => "Normal"
};
if (obj is Txn { Amount: > 0 } txn) { ... }
```

Also: type patterns, relational patterns, list patterns `[1, .., 9]`, and `is not null`.

Q18. What are generics and constraints?

Generics give type safety and avoid boxing with reusable code. Constraints restrict type arguments: `where T : class`, `struct`, `new()`, `notnull`, a base class, or an interface.

```
public interface IRepository<T> where T : class, IEntity {
    Task<T?> GetAsync(int id); Task AddAsync(T e);
}
```

Q19. What is covariance and contravariance?

Covariance (`out T`) allows a more derived type to be used where a base is expected: `IEnumerable<string>` assigns to `IEnumerable<object>`. Contravariance (`in T`) allows the reverse for inputs: `Action<object>` assigns to `Action<string>`.

Q20. Delegates, Func, Action, Predicate and events.

A delegate is a type-safe function pointer. `Func<T, TResult>` returns a value, `Action<T>` returns void, `Predicate<T>` returns bool. An event is a delegate restricted so outsiders can only subscribe/unsubscribe (`+=`/`-=`), and only the owner can raise it.

```
public event EventHandler<TxnEventArgs>? TxnMatched;
protected virtual void OnMatched(Txn t) => TxnMatched?.Invoke(this, new TxnEventArgs(t));
```

> **Interview tip:** Event subscriptions keep subscribers alive; forgetting to unsubscribe is a classic memory leak.

Q21. What are lambda expressions and closures in C#? Any gotcha?

Lambdas are anonymous functions; they can capture outer variables (closures), which the compiler hoists into a hidden class. Gotcha: capturing a variable that later changes, or capturing large objects keeps them alive longer.

Q22. Exception handling best practices.

- Catch specific exceptions; don't swallow them.
- Use `throw;` not `throw ex;` (the latter resets the stack trace).
- Use `finally` or `using` for cleanup.
- Don't use exceptions for normal flow (use `TryParse`, result objects).
- Exception filters: `catch (SqlException ex) when (ex.Number == 1205)` for deadlock retry.
- Custom exceptions for domain errors (`InsufficientBalanceException`).

Q23. What is IDisposable and the using statement?

`IDisposable.Dispose()` releases unmanaged resources (connections, file handles, streams) deterministically. `using` guarantees Dispose even if an exception occurs. `IAsyncDisposable` with `await using` for async cleanup.

```
await using var conn = new SqlConnection(cs);
await conn.OpenAsync();
```

Q24. Dispose vs Finalize.

Finalizer (`~ClassName`) runs non-deterministically on the finalizer thread when GC collects the object, delaying collection by a generation. Dispose is called explicitly. Dispose pattern: `Dispose()` releases resources and calls `GC.SuppressFinalize(this)`. Prefer `SafeHandle` over writing finalizers.

Q25. How does the .NET Garbage Collector work?

Generational, mark-and-compact. Gen 0 (new, short-lived objects, collected often), Gen 1 (buffer), Gen 2 (long-lived). Objects that survive a collection get promoted. LOH holds large objects and is not compacted by default. Server GC (per-core heaps, higher throughput) is the default for ASP.NET Core; Workstation GC for desktop. Reduce pressure with pooling (`ArrayPool<T>`), `Span<T>`, and avoiding unnecessary allocations.

Q26. What are Span<T> and Memory<T>?

`Span<T>` is a stack-only (`ref struct`) view over contiguous memory (array, stack, native) that allows slicing without allocation. `Memory<T>` is the heap-storable version usable across `await`. Great for parsing fixed-width files such as settlement/ISO formats without creating substrings.

```
ReadOnlySpan<char> line = "4111111111111111000000012500";
var pan = line[..16]; var amt = decimal.Parse(line.Slice(16, 12)) / 100m;
```

Q27. What are indexers, properties and init-only setters?

Properties encapsulate fields with get/set logic. `init` (C# 9) allows setting only during object initialisation, enabling immutable objects. `required` (C# 11) forces callers to set a member. Indexers let objects be accessed like arrays: `public Txn this[string rrn] => _map[rrn];`

Q28. What is reflection? When to use it?

Inspecting/creating types and invoking members at runtime (`typeof(T).GetProperties()`). Used by serializers, ORMs and DI containers. It's slow and bypasses compile-time checks; prefer generics, source generators, or cache reflection results. Use case: mapping Excel columns to entity properties by attribute.

Q29. What are attributes?

Metadata attached to code (`[Required]`, `[HttpGet]`, `[JsonPropertyName]`). Read via reflection. Custom: `class ExcelColumnAttribute : Attribute { public string Name { get; } ... }`.

2. OOP and SOLID

Q30. Explain the four pillars of OOP with examples.

- **Encapsulation**: hide state behind methods; `Account.Withdraw()` validates balance instead of public `Balance` setter.
- **Abstraction**: expose what, hide how; `IPaymentGateway.ChargeAsync()`.
- **Inheritance**: reuse and specialise; `CardTxn : Transaction`.
- **Polymorphism**: same call, different behaviour; `IReconRule.Apply()` for many rule types.

Q31. Explain SOLID with real-world C# examples.

- **S** - Single Responsibility: `FileParser` parses; `TxnRepository` saves; `ReconEngine` matches. One reason to change each.
- **O** - Open/Closed: add a new matching rule as a new `IMatchRule` class rather than editing a giant `switch`.
- **L** - Liskov Substitution: subtypes must work wherever the base is used. A `ReadOnlyRepository` that throws in `Add()` violates LSP; split interfaces instead.
- **I** - Interface Segregation: `IReader` and `IWriter` rather than one fat `IRepository` with 20 methods.
- **D** - Dependency Inversion: high-level code depends on abstractions (`INotifier`), injected by DI, not on `new SmtpClient()`.

```
public interface IMatchRule { bool IsMatch(Txn a, Txn b); }
public class RrnRule : IMatchRule { public bool IsMatch(Txn a, Txn b) => a.Rrn == b.Rrn; }
public class AmountRule : IMatchRule { public bool IsMatch(Txn a, Txn b) => a.Amount == b.Amount; }
public class ReconEngine(IEnumerable<IMatchRule> rules) {
    public bool Match(Txn a, Txn b) => rules.All(r => r.IsMatch(a, b));
}
```

Q32. Composition vs inheritance?

Prefer composition ("has-a"): build behaviour from injected collaborators. Inheritance creates tight coupling and fragile base classes. Use inheritance for true "is-a" relationships with stable base behaviour.

Q33. What are DRY, KISS and YAGNI?

Don't Repeat Yourself (single source of truth), Keep It Simple, You Aren't Gonna Need It (don't build speculative features). Balance DRY against coupling: two pieces of code that look the same but change for different reasons can stay separate.

3. Collections and LINQ

Q34. Array vs List vs LinkedList vs Dictionary vs HashSet.

- Array: fixed size, O(1) index.
- `List<T>`: dynamic array, O(1) amortised add, O(n) insert/remove in middle.
- `LinkedList<T>`: O(1) insert/remove with a node reference, no index.
- `Dictionary<K,V>`: hash table, O(1) average lookup.
- `HashSet<T>`: unique values, O(1) contains; set operations.
- `SortedDictionary`/`SortedSet`: red-black tree, O(log n), ordered.
- `Queue<T>`, `Stack<T>`, `PriorityQueue<TElement, TPriority>` (.NET 6).

Q35. IEnumerable vs ICollection vs IList vs IQueryable.

`IEnumerable<T>`: forward-only iteration, in-memory LINQ (delegates). `ICollection<T>`: adds Count, Add, Remove. `IList<T>`: adds index access. `IQueryable<T>`: builds an **expression tree** translated by a provider (EF Core to SQL) and executed at the database.

> **Interview tip:** Classic bug: `repo.GetAll()` returning `IEnumerable` then calling `.Where()` pulls the entire table into memory. Filter on `IQueryable` before materialising.

Q36. What is deferred execution in LINQ?

The query runs only when enumerated (`foreach`, `ToList`, `Count`, `First`). Enumerating twice runs it twice (two DB calls). Materialise with `ToList()` when you need to reuse results.

```
var q = list.Where(x => x > 2);   // nothing runs
list.Add(10);
q.Count();                        // includes 10, runs now
```

Q37. First vs FirstOrDefault vs Single vs SingleOrDefault.

`First` throws if none; `FirstOrDefault` returns default. `Single` throws if none OR more than one; `SingleOrDefault` throws if more than one. Use `Single` when uniqueness is a business rule (lookup by primary key).

Q38. Select vs SelectMany.

`Select` projects each element to one result (can give a list of lists). `SelectMany` flattens: `batches.SelectMany(b => b.Txns)` gives all transactions.

Q39. Write common LINQ queries asked in interviews.

```
// Group by merchant, total amount, top 5
var top = txns.GroupBy(t => t.MerchantId)
              .Select(g => new { Merchant = g.Key, Total = g.Sum(x => x.Amount), Count = g.Count() })
              .OrderByDescending(x => x.Total).Take(5);

// Second highest amount
var second = txns.Select(t => t.Amount).Distinct().OrderByDescending(a => a).Skip(1).FirstOrDefault();

// Duplicates by RRN
var dups = txns.GroupBy(t => t.Rrn).Where(g => g.Count() > 1).Select(g => g.Key);

// Unmatched: in bank file but not in switch (left anti-join)
var switchRrns = switchTxns.Select(s => s.Rrn).ToHashSet();
var unmatched = bankTxns.Where(b => !switchRrns.Contains(b.Rrn));

// Inner join
var joined = from b in bankTxns join s in switchTxns on b.Rrn equals s.Rrn
             select new { b.Rrn, Diff = b.Amount - s.Amount };

// Left join
var left = from b in bankTxns
           join s in switchTxns on b.Rrn equals s.Rrn into gj
           from s in gj.DefaultIfEmpty()
           select new { b.Rrn, Status = s == null ? "MISSING_IN_SWITCH" : "FOUND" };

// Word frequency
var freq = text.Split(' ', StringSplitOptions.RemoveEmptyEntries)
               .GroupBy(w => w.ToLower()).ToDictionary(g => g.Key, g => g.Count());
```

Q40. Any() vs Count() > 0?

`Any()` stops at the first element; `Count()` may enumerate everything (for IEnumerable). In EF Core, `Any()` becomes `EXISTS`, which is more efficient.

Q41. What is yield return?

Creates an iterator lazily, producing items one at a time. Useful for streaming large files without loading everything.

```
IEnumerable<string> ReadLines(string path) {
    using var r = new StreamReader(path);
    string? line;
    while ((line = r.ReadLine()) != null) yield return line;
}
```

Q42. Dictionary vs ConcurrentDictionary vs Hashtable.

`Hashtable` is non-generic (boxing), legacy. `Dictionary<K,V>` is fast but not thread-safe for writes. `ConcurrentDictionary` is thread-safe with fine-grained locking and atomic helpers like `GetOrAdd`, `AddOrUpdate`. Note: the value factory in `GetOrAdd` can run more than once under contention; wrap in `Lazy<T>` if it must run once.

4. Async, Multithreading and Concurrency

Q43. What are async and await? How do they work internally?

`async` lets a method use `await`. The compiler turns it into a **state machine**. At `await` on an incomplete Task, the method returns to the caller and the thread goes back to the pool; when the I/O completes, a continuation resumes the method. For I/O-bound work there is no thread blocked while waiting, which is why ASP.NET Core scales.

Q44. Task vs Thread.

A Thread is an OS thread (expensive: ~1MB stack). A Task is a promise of work, usually scheduled on the ThreadPool, composable with `await`, `WhenAll`, cancellation and exceptions. Use Tasks; create raw threads only for special long-running dedicated work.

Q45. Why is async void bad?

Exceptions can't be caught by the caller (they crash the process via the SynchronizationContext), and the caller can't await completion. Use `async Task`. Only exception: event handlers.

Q46. What is a deadlock with .Result / .Wait()?

Blocking on async code (`GetAsync().Result`) in a context with a SynchronizationContext (old ASP.NET, WinForms, WPF) deadlocks: the continuation needs the blocked thread. ASP.NET Core has no SynchronizationContext, so it won't deadlock there, but blocking still causes **thread-pool starvation** under load. Rule: async all the way.

Q47. What is ConfigureAwait(false)?

It tells the continuation not to resume on the captured context. Recommended in library code. In ASP.NET Core app code it makes no difference.

Q48. Task.WhenAll vs Task.WhenAny vs Parallel.ForEach.

`WhenAll` awaits several tasks concurrently (I/O fan-out). `WhenAny` completes when the first finishes (timeouts, fastest-source). `Parallel.ForEach` / `Parallel.ForEachAsync` (.NET 6) for CPU-bound or throttled parallel work with `MaxDegreeOfParallelism`.

```
var tasks = bankIds.Select(id => client.GetSettlementAsync(id, ct));
var results = await Task.WhenAll(tasks);

await Parallel.ForEachAsync(files, new ParallelOptions { MaxDegreeOfParallelism = 4, CancellationToken =
    ct },
    async (file, token) => await ProcessFileAsync(file, token));
```

Q49. How do you limit concurrency (e.g. max 5 parallel API calls)?

`SemaphoreSlim`:

```
var gate = new SemaphoreSlim(5);
var tasks = items.Select(async i => {
    await gate.WaitAsync(ct);
    try { return await CallApiAsync(i, ct); } finally { gate.Release(); }
});
await Task.WhenAll(tasks);
```

Q50. What is a CancellationToken and how do you use it?

A cooperative cancellation signal. Pass it through all async calls; ASP.NET Core provides `HttpContext.RequestAborted` (bind `CancellationToken ct` in the action). Use `CancellationTokenSource.CancelAfter` for timeouts. Check `ct.ThrowIfCancellationRequested()` in loops.

Q51. lock, Monitor, Mutex, Semaphore, Interlocked.

- `lock` (Monitor): mutual exclusion within a process; cannot `await` inside.
- `SemaphoreSlim`: limit N concurrent entries; supports `WaitAsync` (use for async locking).
- `Mutex`: cross-process lock (slower).
- `Interlocked`: atomic operations on simple values (`Interlocked.Increment(ref count)`).
- `ReaderWriterLockSlim`: many readers, one writer.
- .NET 9 introduces `System.Threading.Lock` type.

Q52. What is a race condition? Give an example and fix.

Two threads read-modify-write shared data without synchronisation, losing updates.

```
int count = 0;
Parallel.For(0, 100000, _ => count++);                    // wrong result
Parallel.For(0, 100000, _ => Interlocked.Increment(ref count)); // correct
```

In distributed systems (multiple API instances), in-memory locks don't help: use DB transactions, row locks, optimistic concurrency (rowversion) or distributed locks (Redis).

Q53. ValueTask vs Task?

`ValueTask` avoids a heap allocation when the result is often available synchronously (cache hits). Rules: await it only once, don't await concurrently. Default to Task.

Q54. What are Channels in .NET?

`System.Threading.Channels` is a high-performance async producer/consumer queue. Bounded channels give backpressure. Ideal for a background pipeline: reader parses a file into the channel, N consumers validate and bulk-insert.

```
var ch = Channel.CreateBounded<Txn>(10_000);
_ = Task.Run(async () => { await foreach (var t in parser.ReadAsync(ct)) await ch.Writer.WriteAsync(t,
    ct); ch.Writer.Complete(); });
await foreach (var t in ch.Reader.ReadAllAsync(ct)) { /* batch and save */ }
```

Q55. What is IAsyncEnumerable?

Asynchronous streams: `await foreach`. Lets you stream DB rows or file lines as they arrive. EF Core supports `AsAsyncEnumerable()`; ASP.NET Core can stream JSON results.

5. ASP.NET Core Fundamentals

Q56. What is .NET Core / modern .NET and how is it different from .NET Framework?

Modern .NET (Core, 5, 6, 8 LTS, 9, 10) is cross-platform (Windows, Linux, macOS), open source, high performance, container-friendly, side-by-side versions, built-in DI, and modular via NuGet. .NET Framework is Windows-only, ships with Windows, and only gets maintenance. ASP.NET Core runs on the Kestrel server and can be hosted behind IIS, Nginx, or in containers.

Q57. Explain Program.cs in minimal hosting (.NET 6+).

```
var builder = WebApplication.CreateBuilder(args);       // config, logging, DI container
builder.Services.AddControllers();
builder.Services.AddDbContext<AppDb>(o => o.UseSqlServer(builder.Configuration.GetConnectionString(
    "Db")));
builder.Services.AddScoped<IReconService, ReconService>();
builder.Services.AddAuthentication(JwtBearerDefaults.AuthenticationScheme).AddJwtBearer(...);
var app = builder.Build();                              // middleware pipeline below
app.UseExceptionHandler("/error");
app.UseHttpsRedirection();
app.UseRouting();
app.UseCors("ui");
app.UseAuthentication();
app.UseAuthorization();
app.MapControllers();
app.Run();
```

Q58. What is middleware? Why does order matter?

Middleware are components in a pipeline; each can handle the request, pass it to `next`, and act on the response on the way back. Order matters: exception handling first, auth before authorization, CORS before auth endpoints, static files early to short-circuit.

```
public class CorrelationIdMiddleware(RequestDelegate next) {
    public async Task InvokeAsync(HttpContext ctx) {
        var id = ctx.Request.Headers["X-Correlation-Id"].FirstOrDefault() ?? Guid.NewGuid().ToString();
        ctx.Response.Headers["X-Correlation-Id"] = id;
        using (LogContext.PushProperty("CorrelationId", id)) await next(ctx);
    }
}
app.UseMiddleware<CorrelationIdMiddleware>();
```

Q59. app.Use vs app.Run vs app.Map.

`Use` adds middleware that can call next. `Run` is terminal (no next). `Map` branches the pipeline by path. `MapWhen` branches by a condition.

Q60. Middleware vs Filters.

Middleware works on every HTTP request, no knowledge of MVC. Filters run inside the MVC pipeline with access to action context, model binding results and action arguments. Filter order: Authorization -> Resource -> Action -> (action executes) -> Exception -> Result.

Q61. Explain dependency injection and service lifetimes.

DI supplies dependencies from a container instead of `new`. Lifetimes:

- **Transient**: new instance every time requested (lightweight, stateless).
- **Scoped**: one per HTTP request (DbContext, unit of work).
- **Singleton**: one for the app lifetime (cache, config, HttpClient factory). Must be thread-safe.

Interview tip: Captive dependency: injecting a Scoped service into a Singleton keeps it alive forever (DbContext shared across threads, bugs). ASP.NET Core validates scopes in Development. In a singleton/background service, create a scope with `IServiceScopeFactory`.

Q62. How do you use a scoped DbContext inside a BackgroundService?

```
public class ReconWorker(IServiceScopeFactory scopes, ILogger<ReconWorker> log) : BackgroundService {
    protected override async Task ExecuteAsync(CancellationToken ct) {
        using var timer = new PeriodicTimer(TimeSpan.FromMinutes(5));
        while (await timer.WaitForNextTickAsync(ct)) {
            using var scope = scopes.CreateScope();
            var svc = scope.ServiceProvider.GetRequiredService<IReconService>();
            try { await svc.RunAsync(ct); } catch (Exception ex) { log.LogError(ex, "Recon run failed"); }
        }
    }
}
```

Q63. What are keyed services (.NET 8)?

Register multiple implementations of an interface with keys and resolve by key: `AddKeyedScoped<IGateway, VisaGateway>("visa")` then `[FromKeyedServices("visa")] IGateway g`.

Q64. How does configuration work? IOptions vs IOptionsSnapshot vs IOptionsMonitor.

Configuration providers are layered: appsettings.json, appsettings.{Environment}.json, user secrets, environment variables, command line, Azure Key Vault (later overrides earlier). Bind to classes with the Options pattern.

- `IOptions<T>`: singleton, read once.
- `IOptionsSnapshot<T>`: scoped, re-read per request.
- `IOptionsMonitor<T>`: singleton with change notifications.

```
builder.Services.AddOptions<SftpSettings>().Bind(builder.Configuration.GetSection("Sftp"))
    .ValidateDataAnnotations().ValidateOnStart();
```

Q65. How do you manage environments and secrets?

`ASPNETCORE_ENVIRONMENT` (Development/Staging/Production) selects appsettings files and behaviours. Never commit secrets: use User Secrets locally, and Key Vault / environment variables / secret managers in production.

Q66. What is Kestrel? Why use a reverse proxy?

Kestrel is the cross-platform built-in web server. A reverse proxy (IIS, Nginx, YARP) adds TLS termination, load balancing, static file caching, request filtering. In-process IIS hosting runs the app inside the IIS worker process for better performance.

Q67. Controllers vs Minimal APIs.

Controllers: attribute routing, filters, model binding conventions; better structure for big APIs. Minimal APIs: less ceremony, slightly faster, endpoint filters and route groups; good for microservices.

```
var g = app.MapGroup("/api/txns").RequireAuthorization();
g.MapGet("/{id:long}", async (long id, AppDb db) =>
    await db.Txns.FindAsync(id) is { } t ? Results.Ok(t) : Results.NotFound());
```

Q68. Explain routing and route constraints.

Attribute routing: `[Route("api/[controller]")]`, `[HttpGet("{id:int}")]`. Constraints: `int`, `guid`, `alpha`, `min(1)`, `regex(...)`. Conventional routing is mostly used in MVC with views.

Q69. What is model binding and [FromBody] / [FromQuery] / [FromRoute] / [FromHeader]?

Model binding maps request data to action parameters. With `[ApiController]`, complex types default to body, simple types to route/query. Only one parameter can be read from the body.

Q70. What does [ApiController] do?

Automatic 400 responses with ProblemDetails on invalid ModelState, binding source inference, attribute routing requirement, and ProblemDetails for error status codes.

Q71. How do you validate input?

DataAnnotations (`[Required]`, `[Range]`, `[StringLength]`) or FluentValidation for complex rules (cross-field, async DB checks). Always validate on the server regardless of client validation.

```
public class PaymentValidator : AbstractValidator<PaymentRequest> {
    public PaymentValidator() {
        RuleFor(x => x.Amount).GreaterThan(0).LessThanOrEqualTo(200000);
        RuleFor(x => x.Rrn).NotEmpty().Length(12).Matches("^[0-9]+$");
    }
}
```

Q72. IActionResult vs ActionResult<T> vs TypedResults.

`IActionResult` returns any result. `ActionResult<T>` allows returning `T` or a status result, and documents the type for Swagger. Minimal APIs use `Results`/`TypedResults` (`Results<Ok<Txn>, NotFound>`).

Q73. How do you implement global exception handling?

.NET 8: implement `IExceptionHandler` and return ProblemDetails; or custom middleware. Map domain exceptions to status codes, log with correlation ID, never leak stack traces in production.

```
public class GlobalExceptionHandler(ILogger<GlobalExceptionHandler> log) : IExceptionHandler {
    public async ValueTask<bool> TryHandleAsync(HttpContext ctx, Exception ex, CancellationToken ct) {
        var (status, title) = ex switch {
            ValidationException => (400, "Validation failed"),
            NotFoundException => (404, "Not found"),
            _ => (500, "Unexpected error") };
        log.LogError(ex, "Unhandled exception");
        ctx.Response.StatusCode = status;
        await ctx.Response.WriteAsJsonAsync(new ProblemDetails { Status = status, Title = title,
            Extensions = { ["traceId"] = ctx.TraceIdentifier } }, ct);
        return true;
    }
}
builder.Services.AddExceptionHandler<GlobalExceptionHandler>(); builder.Services.AddProblemDetails();
app.UseExceptionHandler();
```

Q74. How should HttpClient be used? What is IHttpClientFactory?

Creating `new HttpClient()` per request exhausts sockets (TIME_WAIT); a single static instance ignores DNS changes. `IHttpClientFactory` pools handlers and rotates them. Use typed clients and add resilience (retry, timeout, circuit breaker) with Polly / `Microsoft.Extensions.Http.Resilience`.

```
builder.Services.AddHttpClient<NpciClient>(c => c.BaseAddress = new Uri(cfg["Npci:Url"]!))
    .AddStandardResilienceHandler();
```

Q75. What is API versioning and how do you implement it?

Versions allow breaking changes without breaking clients: URL (`/api/v2/txns`), query string, or header. Use the `Asp.Versioning.Mvc` package with `[ApiVersion("2.0")]`. Deprecate old versions with sunset headers.

Q76. How do you document APIs?

OpenAPI/Swagger (Swashbuckle or the built-in `Microsoft.AspNetCore.OpenApi` in .NET 9 with Scalar UI). Add XML comments, response types (`[ProducesResponseType]`), and auth configuration.

Q77. What is caching in ASP.NET Core?

- In-memory (`IMemoryCache`): per instance, fast.
- Distributed (`IDistributedCache` with Redis/SQL): shared by all instances.
- `HybridCache` (.NET 9): L1 memory + L2 distributed, stampede protection.
- Response caching / Output caching (.NET 7+) for HTTP responses.

Patterns: cache-aside, set expirations, invalidate on writes, avoid caching per-user sensitive data in shared caches.

Q78. How do you implement rate limiting?

.NET 7+ built-in middleware: fixed window, sliding window, token bucket, concurrency limiters, partitioned by user/IP.

```
builder.Services.AddRateLimiter(o => o.AddFixedWindowLimiter("api", x => { x.PermitLimit = 100; x.Window
    = TimeSpan.FromMinutes(1); }));
app.UseRateLimiter();  app.MapControllers().RequireRateLimiting("api");
```

Q79. What are health checks?

`AddHealthChecks().AddSqlServer(cs).AddRedis(...)` and `app.MapHealthChecks("/health")`. Used by load balancers/Kubernetes liveness and readiness probes.

Q80. How do you log in ASP.NET Core?

`ILogger<T>` with structured logging (message templates, not string interpolation): `log.LogInformation("Recon {BatchId} matched {Count}", id, n)`. Providers: Serilog, Seq, Application Insights, ELK. Use log levels properly, correlation IDs, and never log PAN, CVV or passwords (PCI DSS: mask PAN to first 6/last 4).

6. Security

Q81. Authentication vs authorization.

Authentication: who are you (verify identity). Authorization: what are you allowed to do. In the pipeline `UseAuthentication()` comes before `UseAuthorization()`.

Q82. How does JWT authentication work? Configure it.

Client logs in, server issues a signed token with claims (sub, roles, exp). Client sends `Authorization: Bearer <token>`; server validates signature, issuer, audience and expiry without a DB call (stateless).

```
builder.Services.AddAuthentication(JwtBearerDefaults.AuthenticationScheme).AddJwtBearer(o => {
    o.TokenValidationParameters = new() {
        ValidateIssuer = true, ValidIssuer = cfg["Jwt:Issuer"],
        ValidateAudience = true, ValidAudience = cfg["Jwt:Audience"],
        ValidateLifetime = true, ClockSkew = TimeSpan.FromSeconds(30),
        ValidateIssuerSigningKey = true,
        IssuerSigningKey = new SymmetricSecurityKey(Encoding.UTF8.GetBytes(cfg["Jwt:Key"]!)) };
});
```

A JWT is signed, not encrypted: never put sensitive data in it.

Q83. How do refresh tokens work and how do you revoke JWTs?

Access token: short-lived (5-15 min). Refresh token: long-lived, random, stored hashed in DB, rotated on every use (detect reuse = theft). Revocation: delete/flag refresh tokens; for immediate access-token revocation keep a deny-list by `jti` in Redis or keep token lifetime short.

Q84. Role-based vs policy-based vs claims-based authorization.

```
[Authorize(Roles = "Admin")]
builder.Services.AddAuthorization(o => {
    o.AddPolicy("CanApproveRecon", p => p.RequireClaim("permission", "recon.approve"));
    o.AddPolicy("HighValue", p => p.Requirements.Add(new MaxAmountRequirement(1_000_000)));
});
[Authorize(Policy = "CanApproveRecon")]
```

Custom requirements use an `AuthorizationHandler<T>`; resource-based authorization checks the specific record (e.g. user's branch).

Q85. OAuth 2.0 vs OpenID Connect.

OAuth 2.0 is for delegated authorization (access tokens for APIs). OIDC is an identity layer on top that adds the ID token and user info. Authorization Code flow with PKCE for SPAs/mobile; Client Credentials for service-to-service.

Q86. How do you prevent SQL injection, XSS and CSRF?

- SQL injection: parameterised queries, EF Core / Dapper parameters, never concatenate input; least-privilege DB user.
- XSS: Razor encodes output by default; CSP headers; sanitise HTML input.
- CSRF: anti-forgery tokens for cookie-based auth; SameSite cookies. Pure bearer-token APIs are not vulnerable to CSRF in the classic way.

Also: HTTPS + HSTS, security headers, input validation, secrets in a vault.

Q87. How do you store passwords?

Never encrypt or plain-hash. Use a slow salted hash: ASP.NET Core Identity's `PasswordHasher` (PBKDF2), or bcrypt/Argon2.

Q88. What is the Data Protection API?

ASP.NET Core's API for encrypting data like auth cookies and anti-forgery tokens with key rotation. In a web farm, keys must be shared (Redis, blob storage) or users get logged out across instances.

7. Entity Framework Core and Data Access

Q89. What is EF Core? Code-first vs database-first.

An ORM mapping C# classes to tables, with LINQ queries, change tracking and migrations. Code-first: model in C#, migrations create schema. Database-first: scaffold entities from existing DB (`dotnet ef dbcontext scaffold`). In finance systems with DBA-owned schemas, database-first plus stored procedures is common.

Q90. What are migrations?

Versioned schema changes generated from the model: `dotnet ef migrations add AddSettlementTable`, `dotnet ef database update`. In production, generate idempotent SQL scripts (`migrations script --idempotent`) for DBA review rather than auto-migrating at startup.

Q91. What is change tracking? What does AsNoTracking do?

DbContext tracks loaded entities and generates UPDATEs for changes on `SaveChanges`. `AsNoTracking()` skips tracking for read-only queries: faster and less memory. Use it for reports and GET endpoints.

Q92. What is the N+1 problem and how do you fix it?

Loading a list then lazily querying related data per row = 1 + N queries. Fixes: eager loading with `Include`, projection with `Select` into DTOs (best: only needed columns), or split queries.

```
var dto = await db.Batches.AsNoTracking()
    .Where(b => b.Date == date)
    .Select(b => new BatchDto(b.Id, b.Txns.Count, b.Txns.Sum(t => t.Amount)))
    .ToListAsync(ct);
```

Q93. Eager vs lazy vs explicit loading.

Eager: `Include/ThenInclude` in the same query. Lazy: navigation loaded on first access (needs proxies; causes N+1). Explicit: `db.Entry(b).Collection(x => x.Txns).LoadAsync()`.

Q94. What is AsSplitQuery and the cartesian explosion problem?

Including multiple collections in one query multiplies rows (Batch x Txns x Exceptions). `AsSplitQuery()` sends separate SQL queries per collection.

Q95. How do you call stored procedures from EF Core and Dapper?

```
var rows = await db.ReconResults
    .FromSqlInterpolated($"EXEC dbo.usp_GetReconResults @Date={date}, @BankId={bankId}")
    .AsNoTracking().ToListAsync(ct);

await db.Database.ExecuteSqlInterpolatedAsync($"EXEC dbo.usp_RunRecon {date}");

// Dapper
using var conn = new SqlConnection(cs);
var list = await conn.QueryAsync<ReconRow>("dbo.usp_GetReconResults",
    new { Date = date, BankId = bankId }, commandType: CommandType.StoredProcedure, commandTimeout: 300);
```

Interpolated versions are parameterised (safe); `FromSqlRaw` with concatenation is not.

Q96. EF Core vs Dapper vs ADO.NET?

EF Core: productivity, change tracking, migrations, LINQ. Dapper: micro-ORM, near ADO.NET speed, full SQL control, great for reports and SP-heavy apps. ADO.NET: lowest level, e.g. `SqlBulkCopy`. Many projects mix EF Core for CRUD and Dapper for heavy reads.

Q97. How do you insert 1 million rows fast?

`SqlBulkCopy` (or EFCore.BulkExtensions), batching (e.g. 10,000-50,000 per batch), streaming via `IDataReader`, table-valued parameters into a staging table then a set-based MERGE, minimal indexes during load, `TableLock` option. Never `SaveChanges` per row.

```
using var bulk = new SqlBulkCopy(conn, SqlBulkCopyOptions.TableLock, tx) {
    DestinationTableName = "stg.BankTxn", BatchSize = 20000, BulkCopyTimeout = 0 };
bulk.ColumnMappings.Add("Rrn", "Rrn"); bulk.ColumnMappings.Add("Amount", "Amount");
await bulk.WriteToServerAsync(dataTable, ct);
```

Q98. How do you handle transactions in EF Core?

`SaveChanges` is transactional by default. For multiple SaveChanges or mixing raw SQL: `await using var tx = await db.Database.BeginTransactionAsync(ct); ... await tx.CommitAsync(ct);`. For distributed operations across services, use the Outbox pattern/sagas instead of distributed transactions.

Q99. How do you handle concurrency conflicts?

Optimistic concurrency: a `rowversion` column (`[Timestamp]`). EF adds it to the WHERE clause on update; if 0 rows affected it throws `DbUpdateConcurrencyException`. Catch it, reload, merge or tell the user. Pessimistic: `UPDLOCK, ROWLOCK` hints in SQL for critical balance updates.

Q100. What is DbContext pooling and is DbContext thread-safe?

DbContext is **not** thread-safe; don't run two queries in parallel on the same instance (use one per task or `IDbContextFactory`). `AddDbContextPool` reuses instances to reduce allocation cost.

Q101. What are global query filters?

Filters applied automatically to all queries for an entity: soft delete (`HasQueryFilter(t => !t.IsDeleted)`) or multi-tenancy (`t.TenantId == _tenantId`). Bypass with `IgnoreQueryFilters()`.

Q102. Repository and Unit of Work patterns - needed with EF Core?

DbContext already is a unit of work and DbSet a repository. A thin repository can still help to hide query logic, enforce aggregate boundaries, and make tests easier; avoid a generic repository that just wraps DbSet and returns IQueryable everywhere.

Q103. What are compiled queries and other EF performance tips?

`EF.CompileAsyncQuery` caches translation for hot queries. Other tips: projection, AsNoTracking, pagination with keyset (`WHERE Id > @lastId`) instead of large OFFSET, indexes on filter columns, avoid client evaluation, `ExecuteUpdate/ExecuteDelete` (EF 7) for set-based updates without loading entities.

```
await db.Txns.Where(t => t.BatchId == id && t.Status == "PENDING")
             .ExecuteUpdateAsync(s => s.SetProperty(t => t.Status, "EXPIRED"), ct);
```

8. Design Patterns and Architecture

Q104. Which design patterns have you used? Explain a few with code.

- **Singleton**: one instance (prefer DI singleton lifetime over static).
- **Factory**: create objects by type without exposing `new` logic.
- **Strategy**: choose an algorithm at runtime (matching rules, fee calculators).
- **Repository / Unit of Work**: data access abstraction.
- **Decorator**: add behaviour (caching, logging) around a service.
- **Adapter**: wrap a third-party API in your interface.
- **Observer**: events, pub/sub.
- **Builder**: step-by-step complex object creation (reports, ISO messages).
- **Mediator** (MediatR): decouple request senders from handlers (CQRS).
- **Chain of Responsibility**: middleware pipeline, validation chains.

```
// Strategy + Factory: parse files from different banks
public interface IFileParser { string BankCode { get; } IAsyncEnumerable<Txn> ParseAsync(Stream s); }
public class ParserFactory(IEnumerable<IFileParser> parsers) {
    public IFileParser For(string bank) => parsers.FirstOrDefault(p => p.BankCode == bank)
        ?? throw new NotSupportedException($"No parser for {bank}");
}
// Decorator: caching around a repository
public class CachedRateRepo(IRateRepo inner, IMemoryCache cache) : IRateRepo {
    public Task<decimal> GetRateAsync(string ccy) =>
        cache.GetOrCreateAsync(ccy, e => { e.AbsoluteExpirationRelativeToNow = TimeSpan.FromMinutes(10);
    return inner.GetRateAsync(ccy); })!;
}
```

Q105. Implement a thread-safe singleton.

```
public sealed class AppClock {
    private static readonly Lazy<AppClock> _i = new(() => new AppClock());
    public static AppClock Instance => _i.Value;
    private AppClock() { }
}
```

`Lazy<T>` is thread-safe by default. In ASP.NET Core prefer `AddSingleton`.

Q106. What is Clean Architecture / Onion Architecture?

Layers with dependencies pointing inward: **Domain** (entities, value objects, business rules, no dependencies) <- **Application** (use cases, interfaces like `ITxnRepository`, DTOs, validation) <- **Infrastructure** (EF Core, file system, external APIs implementing interfaces) and **Presentation/API** (controllers). The core is testable and independent of frameworks.

Q107. What is CQRS? Show it with MediatR.

Command Query Responsibility Segregation: separate models/handlers for writes (commands) and reads (queries). Reads can use Dapper/denormalised views; writes use domain model + EF. MediatR also enables pipeline behaviours for validation, logging and transactions.

```
public record ApproveReconCommand(long BatchId, string User) : IRequest<Result>;
public class ApproveReconHandler(AppDb db) : IRequestHandler<ApproveReconCommand, Result> {
    public async Task<Result> Handle(ApproveReconCommand c, CancellationToken ct) {
        var b = await db.Batches.FindAsync([c.BatchId], ct);
        if (b is null) return Result.NotFound();
        b.Approve(c.User);                // domain logic in entity
        await db.SaveChangesAsync(ct);
        return Result.Ok();
    }
}
```

Q108. Monolith vs microservices.

Monolith: single deployable, simple, easier transactions and debugging. Microservices: independently deployable services per business capability, separate databases, independent scaling, but add network failures, eventual consistency, distributed tracing and DevOps complexity. A modular monolith is often the right first step.

Q109. How do microservices communicate?

Synchronous: REST, gRPC (fast, HTTP/2, protobuf, great internal). Asynchronous: message brokers (RabbitMQ, Kafka, Azure Service Bus) for events and decoupling. Use an API Gateway (YARP, Ocelot, Azure APIM) for routing, auth, rate limiting.

Q110. What is the Saga pattern and the Outbox pattern?

Saga: a distributed transaction as a sequence of local transactions with compensating actions on failure (orchestration or choreography). Outbox: write the business change and the event to an Outbox table in the **same DB transaction**; a background worker publishes events, guaranteeing at-least-once delivery. Consumers must be idempotent.

Q111. What is a circuit breaker? Retry with backoff?

Retry with exponential backoff + jitter handles transient failures. A circuit breaker stops calling a failing dependency for a period (open state), then tries again (half-open). Combine with timeouts and fallbacks. In .NET: Polly v8 / `AddStandardResilienceHandler`.

Q112. How do you trace a request across microservices?

Distributed tracing with OpenTelemetry (W3C trace context headers), exported to Jaeger, Zipkin, Application Insights or Grafana. Correlation IDs in structured logs. .NET Aspire provides a dashboard for local development.

Q113. What is Docker and how do you containerise a .NET app?

```
FROM mcr.microsoft.com/dotnet/sdk:8.0 AS build
WORKDIR /src
COPY . .
RUN dotnet publish Api/Api.csproj -c Release -o /app
FROM mcr.microsoft.com/dotnet/aspnet:8.0
WORKDIR /app
COPY --from=build /app .
ENTRYPOINT ["dotnet", "Api.dll"]
```

Multi-stage builds keep the runtime image small. .NET 8 can also `dotnet publish /t:PublishContainer` without a Dockerfile.

Q114. What is gRPC and when would you choose it over REST?

High-performance RPC over HTTP/2 with protobuf contracts, streaming, and strongly typed generated clients. Best for internal service-to-service calls. REST/JSON is better for public APIs and browsers.

Q115. What is SignalR?

Real-time communication library (WebSockets with fallback to SSE/long polling). Hubs push updates to clients: live dashboards, notifications. Scale out with a Redis backplane or Azure SignalR Service.

9. Testing

Q116. How do you unit test in .NET?

xUnit/NUnit/MSTest + Moq/NSubstitute + FluentAssertions. Follow Arrange-Act-Assert; test behaviour, not private methods; mock external dependencies only.

```
[Fact]
public async Task Approve_ReturnsNotFound_WhenBatchMissing() {
    var repo = new Mock<IBatchRepo>();
    repo.Setup(r => r.GetAsync(1, default)).ReturnsAsync((Batch?)null);
    var sut = new ReconService(repo.Object);
    var result = await sut.ApproveAsync(1, "u", default);
    result.IsNotFound.Should().BeTrue();
}
[Theory] [InlineData(0)] [InlineData(-5)]
public void Amount_MustBePositive(decimal amt) =>
    new PaymentValidator().Validate(new PaymentRequest { Amount = amt }).IsValid.Should().BeFalse();
```

Q117. How do you write integration tests for an API?

`WebApplicationFactory<Program>` spins up the app in memory; replace dependencies via `ConfigureTestServices`; use Testcontainers for a real SQL Server in Docker rather than the EF InMemory provider (which doesn't behave like SQL).

Q118. What is the difference between mocks, stubs and fakes?

Stub returns canned data. Mock also verifies interactions (`Verify(x => x.Send(...), Times.Once)`). Fake is a working lightweight implementation (in-memory repository).

10. Coding Questions in C#

Q119. Reverse a string and check palindrome.

```
string Reverse(string s) { var a = s.ToCharArray(); Array.Reverse(a); return new string(a); }
bool IsPalindrome(string s) {
    int i = 0, j = s.Length - 1;
    while (i < j) {
        if (!char.IsLetterOrDigit(s[i])) { i++; continue; }
        if (!char.IsLetterOrDigit(s[j])) { j--; continue; }
        if (char.ToLower(s[i++]) != char.ToLower(s[j--])) return false;
    }
    return true;
}
```

Q120. Count character frequency / first non-repeating character.

```
char? FirstUnique(string s) {
    var freq = new Dictionary<char, int>();
    foreach (var c in s) freq[c] = freq.GetValueOrDefault(c) + 1;
    foreach (var c in s) if (freq[c] == 1) return c;
    return null;
}
```

Q121. Find duplicates in an array.

```
var seen = new HashSet<int>(); var dups = new HashSet<int>();
foreach (var n in arr) if (!seen.Add(n)) dups.Add(n);
```

Q122. FizzBuzz, Fibonacci, prime check, factorial.

```
for (int i = 1; i <= 100; i++) Console.WriteLine(i % 15 == 0 ? "FizzBuzz" : i % 3 == 0 ? "Fizz" : i % 5
    == 0 ? "Buzz" : i.ToString());
long Fib(int n) { long a = 0, b = 1; for (int i = 0; i < n; i++) (a, b) = (b, a + b); return a; }
bool IsPrime(int n) { if (n < 2) return false; for (int i = 2; (long)i * i <= n; i++) if (n % i == 0)
    return false; return true; }
```

Q123. Validate a card number with the Luhn algorithm.

```
bool Luhn(string pan) {
    int sum = 0; bool dbl = false;
    for (int i = pan.Length - 1; i >= 0; i--) {
        int d = pan[i] - '0';
        if (dbl) { d *= 2; if (d > 9) d -= 9; }
        sum += d; dbl = !dbl;
    }
    return sum % 10 == 0;
}
string Mask(string pan) => pan[..6] + new string('*', pan.Length - 10) + pan[^4..];
```

Q124. Implement a simple LRU cache.

```
public class LruCache<K, V>(int capacity) where K : notnull {
    private readonly Dictionary<K, LinkedListNode<(K key, V val)>> map = new();
    private readonly LinkedList<(K key, V val)> list = new();
    public bool TryGet(K k, out V v) {
        if (map.TryGetValue(k, out var node)) { list.Remove(node); list.AddFirst(node); v = node.Value.
    val; return true; }
        v = default!; return false;
    }
    public void Put(K k, V v) {
        if (map.TryGetValue(k, out var node)) { list.Remove(node); map.Remove(k); }
        else if (map.Count == capacity) { map.Remove(list.Last!.Value.key); list.RemoveLast(); }
        map[k] = list.AddFirst((k, v));
    }
}
```

Q125. Write a generic retry helper with exponential backoff.

```
public static async Task<T> RetryAsync<T>(Func<Task<T>> op, int max = 3, CancellationToken ct = default) {
    for (int attempt = 1; ; attempt++) {
        try { return await op(); }
        catch (Exception ex) when (attempt < max && IsTransient(ex)) {
            var delay = TimeSpan.FromMilliseconds(200 * Math.Pow(2, attempt) + Random.Shared.Next(100));
            await Task.Delay(delay, ct);
        }
    }
}
```

Q126. Producer-consumer with BlockingCollection or Channel.

See Channels above. With `BlockingCollection<T>` (sync): producer calls `Add` then `CompleteAdding`; consumers `foreach (var x in bc.GetConsumingEnumerable())`.

11. Real-World Scenario Questions

Q127. Your API is slow in production. How do you investigate?

1) Measure: APM (App Insights/OpenTelemetry) to find slow endpoints and dependency calls. 2) Check the DB: slow queries, missing indexes, execution plans, N+1, parameter sniffing, blocking/deadlocks. 3) Check the app: sync-over-async/thread-pool starvation (`dotnet-counters`), high GC (`dotnet-trace`, `dotnet-dump`), large payloads. 4) Fix: projection + AsNoTracking, pagination, caching, async I/O, bulk operations, compression. 5) Load test (k6, JMeter) to confirm.

Q128. Memory keeps growing in your service. What could cause it?

Static collections/caches without eviction, event handlers not unsubscribed, captive scoped dependencies in singletons, undisposed streams/HttpResponseMessages, large objects in LOH fragmentation, `IMemoryCache` without size limits. Diagnose with `dotnet-counters`, `dotnet-gcdump`/`dotnet-dump` and compare heap snapshots.

Q129. How would you design a file-upload-and-process feature (e.g. 5 lakh-row Excel)?

API accepts the upload (streaming, size/type validation), stores it (blob/disk) and returns 202 Accepted with a job ID. A background worker (BackgroundService/Hangfire/queue) streams-parses the file (ExcelDataReader/Open XML SAX, not loading all to memory), validates rows, bulk-inserts into a staging table with SqlBulkCopy, runs set-based SP logic, writes a rejection report, updates job status. UI polls or receives SignalR updates. Make it idempotent (file hash) and resumable.

Q130. How do you make a payment API idempotent?

Client sends `Idempotency-Key`. Server stores key + request hash + response in a table with a unique constraint. Same key: return the stored response; same key with different payload: 422. Insert the key in the same transaction as the business operation. Also DB unique constraints on business keys (e.g. RRN + STAN + terminal).

Q131. Two users approve the same reconciliation batch at the same time. How do you prevent double processing?

Optimistic concurrency with rowversion, or a conditional update: `UPDATE Batch SET Status='APPROVED' WHERE Id=@id AND Status='PENDING'` and check rows affected = 1. For jobs across multiple servers, use DB-based locks (`sp_getapplock`) or a distributed lock.

Q132. How do you schedule recurring jobs in .NET?

`BackgroundService` + `PeriodicTimer` for simple cases; Hangfire or Quartz.NET for cron schedules, persistence, retries, dashboards and multi-instance safety. In the cloud: Azure Functions timer triggers or Kubernetes CronJobs.

Q133. How would you migrate a .NET Framework app to .NET 8?

Assess with .NET Upgrade Assistant and API portability analysis; move libraries to .NET Standard 2.0 or multi-target first; replace unsupported tech (WCF server -> CoreWCF/gRPC, Web Forms -> rewrite, System.Web -> ASP.NET Core abstractions); migrate incrementally with the strangler-fig pattern (YARP proxying unmigrated routes to the old app); comprehensive regression tests.

Q134. How do you secure an API that handles card data (PCI DSS)?

TLS 1.2+, strong auth (OAuth2/JWT, mTLS between services), never store CVV, tokenise or encrypt PAN at rest (with key management/HSM), mask PAN in logs and UI, least privilege, audit logging, input validation, rate limiting, dependency scanning, secrets in a vault, and network segmentation.

Q135. How would you handle a third-party API that is slow and sometimes fails?

Typed HttpClient with timeout, retry with backoff for transient errors (5xx, 408, 429 honouring Retry-After), circuit breaker, fallback/cached response, bulkhead (limit concurrent calls), async processing via queue if the user doesn't need an immediate answer, and alerts on failure rates.

Q136. How do you deploy with zero downtime?

Blue-green or rolling deployments behind a load balancer, readiness health checks, backward-compatible DB migrations (expand-and-contract: add new column, deploy code, backfill, remove old later), feature flags, and graceful shutdown (`IHostApplicationLifetime`, handling SIGTERM).

Q137. A deadlock is reported between your API and a batch job. What do you do?

Capture deadlock graph (SQL Server system_health Extended Events), identify resources and access order, add appropriate indexes to reduce lock scope, access tables in a consistent order, keep transactions short, use READ COMMITTED SNAPSHOT for readers, batch the job's updates, and add retry logic for error 1205.

12. Quick-Fire Questions

Q138. What is the CLR, CTS, CLS, JIT and IL?

C# compiles to Intermediate Language (IL) in assemblies. The CLR (Common Language Runtime) loads it, JIT-compiles IL to native code at runtime, and provides GC, type safety and exception handling. CTS defines types all .NET languages share; CLS is the subset for cross-language compatibility. Native AOT compiles ahead of time for fast startup.

Q139. What is the difference between an assembly and a namespace?

An assembly is a compiled unit (.dll/.exe) with IL and metadata. A namespace is a logical grouping of types; one assembly can hold many namespaces and vice versa.

Q140. What is `dynamic` vs `var` vs `object`?

`var`: compile-time type inference (statically typed). `object`: base type, needs casting. `dynamic`: type checking deferred to runtime.

Q141. What does the `params` keyword do?

Allows a variable number of arguments: `Sum(params int[] nums)`. C# 13 allows `params` on collections like `Span<T>`.

Q142. What is a tuple / deconstruction?

```
(decimal total, int count) Stats() => (100m, 3); var (t, c) = Stats();
```

Q143. What is `nameof`?

Returns the name of a variable/member as a string, refactoring-safe: `throw new ArgumentNullException(nameof(request));` or `ArgumentNullException.ThrowIfNull(request);`.

Q144. What are primary constructors (C# 12)?

Parameters on the class declaration available throughout the body: `public class ReconService(IRepo repo, ILogger<ReconService> log) { ... }`. Commonly used for DI.

Q145. What are collection expressions (C# 12)?

```
int[] a = [1, 2, 3]; List<string> l = ["x", "y"]; int[] all = [..a, 4];
```

Q146. What is the difference between `IEnumerable.ToList()` and `ToArray()`?

Both materialise. List is resizable; Array is fixed and slightly smaller. Return `IReadOnlyList<T>` from APIs to signal immutability.

Q147. Difference between PUT and PATCH; what are idempotent HTTP methods?

PUT replaces the whole resource; PATCH partially updates. GET, PUT, DELETE, HEAD, OPTIONS are idempotent; POST is not (and PATCH may not be).

Q148. What HTTP status codes should an API return?

200 OK, 201 Created (+Location), 202 Accepted (async job), 204 No Content, 400 Bad Request, 401 Unauthorized (not authenticated), 403 Forbidden (authenticated but no permission), 404 Not Found, 409 Conflict (concurrency/duplicate), 422 Unprocessable Entity, 429 Too Many Requests, 500 Internal Server Error, 503 Service Unavailable.

Q149. What is REST? What makes an API RESTful?

Architectural style: resources identified by URIs, standard HTTP methods, stateless requests, representations (JSON), cacheability, uniform interface. Use nouns (`/api/batches/12/txns`), not verbs; plural names; proper status codes; pagination/filtering via query parameters.

13. OOP Deep Dive: Every Concept and Tricky Questions

Q150. What is a class and what is an object?

A class is a blueprint defining state (fields/properties) and behaviour (methods). An object is an instance of that class in memory with its own state.

Q151. What is a constructor? Types of constructors in C#?

Special method that initialises an object. Types: default (parameterless), parameterised, copy (user-defined), static, private (prevents external instantiation, used in singletons), and primary constructors (C# 12).

Q152. If you define a parameterised constructor, does the compiler still give a default constructor?

No. The implicit parameterless constructor exists only if you define no constructors. `new Txn()` then fails to compile.

Q153. What is constructor chaining?

Calling another constructor of the same class with `this(...)` or the base class with `base(...)`.

```
public Txn(string rrn) : this(rrn, 0m) { }
public Txn(string rrn, decimal amt) : base(DateTime.UtcNow) { Rrn = rrn; Amount = amt; }
```

Q154. In what order do constructors run in inheritance?

Field initialisers of the derived class, then field initialisers of the base, then base constructor body, then derived constructor body. Static constructors: derived static first when the derived type is touched, then base static as needed.

```
class A { public A() { Console.Write("A "); } }
class B : A { public B() { Console.Write("B "); } }
new B();   // A B
```

Q155. Tricky: what happens if you call a virtual method from a base constructor?

The most-derived override runs before the derived constructor body has executed, so derived fields may still be uninitialised (default values). Avoid calling virtual members in constructors.

```
class Base { public Base() { Print(); } public virtual void Print() { } }
class Der : Base { string name = "x"; string late; public Der() { late = "y"; }
  public override void Print() => Console.WriteLine($"{name} {late ?? "null"}"); }
new Der(); // "x null" (field initialiser ran, constructor body not yet)
```

Q156. Can a constructor be private? Why?

Yes: singletons, static factory methods (`Money.From(...)`), classes with only static members, or forcing use of a builder.

Q157. Can a constructor be static and have parameters/access modifiers?

A static constructor cannot have parameters or access modifiers, runs once automatically, and cannot be called directly.

Q158. Can constructors be inherited or overridden?

No. Derived classes must declare their own constructors and call base constructors.

Q159. What is a destructor/finalizer in C#?

`~ClassName()` compiled into `Finalize()`. Called by GC non-deterministically. Cannot be called explicitly, no parameters, no modifiers. Use only for unmanaged resources, with the Dispose pattern.

Q160. What is encapsulation in practice? Example of breaking it.

Hiding internal state and exposing controlled operations. Breaking it: public setters on `Balance`, returning a mutable internal `List<T>` that callers can modify. Return `IReadOnlyCollection<T>` instead.

```
public class Account { private readonly List<Entry> _entries = new(); public decimal Balance { get;
    private set; }
  public IReadOnlyCollection<Entry> Entries => _entries.AsReadOnly();
  public void Debit(decimal amt) { if (amt <= 0 || amt > Balance) throw new InvalidOperationException();
    Balance -= amt; _entries.Add(new(-amt)); } }
```

Q161. Abstraction vs encapsulation - difference?

Abstraction is about exposing only essential behaviour (what), designed via interfaces/abstract classes. Encapsulation is about hiding data and implementation details (how), achieved via access modifiers. Abstraction is at design level; encapsulation at implementation level.

Q162. Types of inheritance and which C# supports?

Single, multilevel, hierarchical are supported for classes. Multiple inheritance of classes is not supported (diamond problem); multiple interface inheritance is. Hybrid can be achieved with interfaces.

Q163. Why doesn't C# support multiple class inheritance?

To avoid the diamond problem (ambiguity when two bases implement the same member), complexity in object layout and constructor order. Interfaces solve most use cases.

Q164. Tricky: Diamond problem with default interface methods - how is it resolved?

If two interfaces provide default implementations of the same member, the class must implement it explicitly, otherwise compile error. Ambiguity must be resolved by the implementer.

Q165. Types of polymorphism in C#?

Compile-time (static): method overloading, operator overloading, generics. Run-time (dynamic): method overriding via virtual/abstract and interfaces.

Q166. Can you overload methods by return type only?

No. Overloads must differ in number, type, order or kind (ref/out/in) of parameters. `ref` vs `out` alone is not enough to distinguish.

Q167. Tricky: which overload is called?

```
void M(object o) => Console.Write("object");
void M(string s) => Console.Write("string");
M(null);         // "string" (most specific)
void N(int x) => Console.Write("int"); void N(long x) => Console.Write("long");
N(5);            // "int"; N(5L) -> "long"; byte b=1; N(b) -> "int" (better conversion)
```

Overload resolution happens at compile time based on static types.

Q168. Tricky: overload resolution with base type variable.

```
void P(Animal a) => Console.Write("Animal"); void P(Dog d) => Console.Write("Dog");
Animal x = new Dog(); P(x);   // "Animal" (static type decides overloads)
```

Overriding uses run-time type; overloading uses compile-time type. Use `dynamic` or virtual dispatch to get "Dog".

Q169. What is operator overloading? Example.

```
public readonly record struct Money(decimal Amount, string Ccy) {
  public static Money operator +(Money a, Money b) => a.Ccy == b.Ccy ? new(a.Amount + b.Amount, a.Ccy) :
    throw new InvalidOperationException("Currency mismatch"); }
```

Some operators must be overloaded in pairs (`==`/`!=`, `<`/`>`).

Q170. What are virtual, override, abstract, sealed and new keywords together?

`virtual`: base provides overridable implementation. `abstract`: no implementation, must override, class must be abstract. `override`: provides new implementation. `sealed override`: stops further overriding. `new`: hides (not overrides) a base member.

Q171. Can an abstract class have a constructor? Can you instantiate it?

It can have constructors (called by derived classes via base) but cannot be instantiated directly.

Q172. Can an abstract class have no abstract methods?

Yes. Marking a class abstract simply prevents instantiation.

Q173. Can an abstract method be private or static?

No. Abstract methods must be overridable, so they can't be private, static, or sealed. (Interfaces can have `static abstract` members since C# 11 for generic math.)

Q174. Can an interface have fields, constructors, private methods?

No instance fields or constructors. Since C# 8: default implementations, private methods, static members, static fields. Since C# 11: static abstract members.

Q175. Explicit interface implementation - what and why?

Implementing a member so it's only accessible through the interface type; used to resolve name clashes or hide members.

```
class FileLogger : ILogger, IAudit { void ILogger.Write(string m) { } void IAudit.Write(string m) { } }
((ILogger)new FileLogger()).Write("x");
```

Q176. Can a struct inherit from a class? Implement interfaces?

Structs can't inherit from classes or structs (they implicitly derive from System.ValueType) but can implement interfaces. Casting a struct to an interface boxes it.

Q177. Can a struct have a parameterless constructor?

Since C# 10, yes (explicit parameterless constructors and field initialisers). `default(T)` still bypasses it and zero-initialises.

Q178. What is the difference between `is-a` and `has-a`?

Is-a = inheritance (`SavingsAccount : Account`). Has-a = composition/aggregation (`Account` has `Customer`).

Q179. Association vs aggregation vs composition.

Association: objects use each other (Teacher-Student). Aggregation: whole-part where parts can live independently (Department-Employee). Composition: strong ownership, parts die with the whole (Order-OrderLines).

Q180. What is coupling and cohesion?

Coupling: degree of dependency between modules (want low). Cohesion: how related responsibilities inside a module are (want high).

Q181. What is method hiding vs shadowing?

In C#, "hiding" with `new` is the term; VB calls it shadowing. Hiding resolves the method by static type.

Q182. Tricky: output with new vs override when calling through base.

```
class A { public virtual void F() => Console.Write("A.F "); public void G() => Console.Write("A.G "); }
class B : A { public override void F() => Console.Write("B.F "); public new void G() => Console.Write("B.
    G "); }
A a = new B(); B b = new B();
a.F(); a.G(); b.F(); b.G();   // B.F A.G B.F B.G
```

Q183. Tricky: what if a derived class method has the same signature without new or override?

It compiles with warning CS0108 and behaves as `new` (hiding).

Q184. Can you override a non-virtual method?

No, compile error. You can only hide it with `new`.

Q185. Can you change the access modifier when overriding?

No, must be the same (except `protected internal` overriding across assemblies becomes `protected`).

Q186. Can you override a property or event?

Yes, if declared virtual/abstract.

Q187. What is a partial class / partial method?

A class split across files, combined at compile time (designer code, source generators). Partial methods declare a signature in one part and optionally implement it in another.

Q188. What is a nested class? Can it access outer private members?

A class declared inside another. It can access private members of the outer class (given an instance), useful for builders and enumerators.

Q189. What is an anonymous type?

Compiler-generated immutable type: `new { Rrn = "1", Amount = 5m }`. Used in LINQ projections; scoped to the method; value-based Equals.

Q190. What is the `this` keyword used for?

Refer to the current instance, pass the instance, chain constructors, declare indexers, define extension methods.

Q191. What is the `base` keyword used for?

Call base class constructors and access overridden or hidden base members (`base.Validate()`).

Q192. What is object slicing? Does it happen in C#?

In C++, copying a derived object into a base by value loses derived data. In C# classes are references, so no slicing; but with structs you can't inherit, so it doesn't apply.

Q193. What is upcasting and downcasting?

Upcasting (Derived to Base) is implicit and safe. Downcasting requires explicit cast and may throw InvalidCastException. Use `is` or `as` patterns.

```
if (shape is Circle c) Console.Write(c.Radius);
var r = shape as Rectangle;   // null if not
```

Q194. `is` vs `as` vs direct cast.

`is` tests (with pattern capture). `as` returns null on failure (reference/nullable types only). Direct cast throws on failure and works with value types and user-defined conversions.

Q195. What are implicit and explicit conversion operators?

```
public static implicit operator decimal(Money m) => m.Amount;
public static explicit operator Money(decimal d) => new(d, "INR");
```

Q196. What is the Liskov principle violation in the Rectangle/Square example?

Square inherits Rectangle but setting Width also sets Height, breaking callers that expect independent dimensions. Model them separately or with immutable shapes.

Q197. What is a God class and how do you fix it?

A class doing too much (thousands of lines). Fix by extracting responsibilities into services following SRP.

Q198. What is an interface marker?

An empty interface used to tag types (e.g. `IAggregateRoot`). Attributes are an alternative.

Q199. When would you choose abstract class over interface in real projects?

When related classes share code and state: e.g. `BaseFileProcessor` with template method `Process()` that calls abstract `ParseLine()`; while each bank's processor implements only parsing.

Q200. What is the Template Method pattern?

Base class defines algorithm skeleton; subclasses override steps.

```
public abstract class FileProcessor { public async Task RunAsync(Stream s) { var rows = Parse(s);
    Validate(rows); await SaveAsync(rows); }
  protected abstract List<Txn> Parse(Stream s); protected virtual void Validate(List<Txn> r) { }
    protected abstract Task SaveAsync(List<Txn> r); }
```

Q201. What is an immutable object and how do you create one in C#?

State can't change after creation: readonly fields, get-only or init properties, no setters, return new instances on change (`with`). Records make this easy. Benefits: thread-safety, safe sharing, predictable.

Q202. What is a value object (DDD) vs entity?

Entity has identity (TxnId) that persists across changes. Value object is defined by its values (Money, Address), immutable and compared by value; records are ideal.

Q203. What is the difference between shallow copy and deep copy in C#?

`MemberwiseClone()` makes a shallow copy (references shared). Deep copy duplicates referenced objects too (manual, serialization, or copy constructors). Records' `with` is shallow.

Q204. What is ICloneable and why is it discouraged?

It doesn't specify shallow or deep and returns object. Prefer explicit copy methods or records.

Q205. What is IComparable vs IComparer?

`IComparable<T>.CompareTo` defines natural ordering inside the type. `IComparer<T>` is an external comparison strategy (sort by amount, by date).

Q206. What is IEquatable<T> and why implement it?

Strongly typed Equals avoiding boxing for structs; used by collections. Implement with `GetHashCode` override.

Q207. Tricky: two records with a list property - are they equal?

```
record R(List<int> Items);
new R(new() { 1 }) == new R(new() { 1 })  // false: List compared by reference
```

Record equality is member-wise using each member's Equals.

Q208. Can records inherit? Can a record inherit from a class?

Records can inherit from records (not from classes, except object). Equality considers the runtime type (EqualityContract).

Q209. What is a sealed record and ToString in records?

Records auto-generate `ToString()` printing properties; you can override or seal it. Sealed records can't be inherited.

Q210. Interview: design a class hierarchy for payments (Card, UPI, NetBanking).

Interface `IPaymentMethod { Task<PaymentResult> PayAsync(PaymentRequest r); }`, implementations per method, a factory resolving by type, shared validation in a base or decorator, results as records. Open/closed: adding Wallet means a new class only.

Q211. Interview: design a parking lot (OOP design).

Classes: ParkingLot (floors), Floor (spots), Spot (type, isFree), Vehicle (type), Ticket (entry time, spot), FeeStrategy (per hour, per vehicle type), EntryGate/ExitGate. Use Strategy for fees and Factory for vehicle types; handle concurrency on spot allocation.

Q212. Interview: design an ATM (OOP design).

ATM with states (Idle, CardInserted, Authenticated, Transaction) using the State pattern; Card, Account, BankService interface, Transaction types (Withdraw, Deposit, BalanceInquiry) via Command pattern; CashDispenser with denominations (Chain of Responsibility for note splitting).

14. C# Tricky Output Questions

Q213. Output: string immutability.

```
string s = "a"; string t = s; s += "b"; Console.Write(t);   // a
```

Q214. Output: string interning and reference equality.

```
string a = "hello"; string b = "hel" + "lo"; string c = new string("hello".ToCharArray());
Console.Write(ReferenceEquals(a, b)); // True (compile-time constant, interned)
Console.Write(ReferenceEquals(a, c)); // False
Console.Write(a == c);                // True (value equality)
```

Q215. Output: object == with strings.

```
object x = "abc"; object y = new string("abc".ToCharArray());
Console.Write(x == y);       // False (object == is reference equality)
Console.Write(x.Equals(y));  // True
```

Q216. Output: struct in a List.

```
struct P { public int X; }
var list = new List<P> { new P() };
// list[0].X = 5;   // compile error: cannot modify return value (it's a copy)
var arr = new P[1]; arr[0].X = 5;  // allowed: array element is a variable
```

Q217. Output: passing reference type by value.

```
void Change(List<int> l) { l.Add(1); l = new List<int> { 99 }; }
var x = new List<int>(); Change(x); Console.Write(x.Count);  // 1
```

The reference is copied; reassigning the parameter doesn't affect the caller, but mutating the object does. With `ref`, the caller would see `{99}`.

Q218. Output: closures in a for loop with lambdas.

```
var actions = new List<Action>();
for (int i = 0; i < 3; i++) actions.Add(() => Console.Write(i));
actions.ForEach(a => a());   // 333
var a2 = new List<Action>(); foreach (var j in new[] { 0, 1, 2 }) a2.Add(() => Console.Write(j));
a2.ForEach(a => a());        // 012 (foreach variable is per-iteration since C# 5)
```

Q219. Output: integer overflow.

```
int max = int.MaxValue; Console.Write(max + 1);          // -2147483648 (unchecked by default)
checked { Console.Write(max + 1); }                      // OverflowException
```

Enable `<CheckForOverflowUnderflow>` or use `checked` for financial counters; or use long/decimal.

Q220. Output: integer division and decimal.

```
Console.Write(5 / 2);     // 2
Console.Write(5 / 2.0);   // 2.5
Console.Write(5m / 2);    // 2.5
Console.Write(-7 / 2);    // -3 (truncates toward zero)
Console.Write(-7 % 3);    // -1
```

Q221. Output: Math.Round banker's rounding.

```
Math.Round(2.5)  // 2
Math.Round(3.5)  // 4
Math.Round(2.5, MidpointRounding.AwayFromZero)  // 3
```

Important for financial rounding rules; be explicit.

Q222. Output: double comparison.

```
Console.Write(0.1 + 0.2 == 0.3);     // False
Console.Write(0.1m + 0.2m == 0.3m);  // True
```

Q223. Output: nullable arithmetic.

```
int? a = null; int? b = 5; Console.Write(a + b == null);   // True (lifted operators)
Console.Write(a > 1 || a <= 1);  // False (comparisons with null are false)
```

Q224. Output: string concatenation with null.

```
string s = null; Console.Write("a" + s + "b");   // ab (null treated as empty)
Console.Write(s?.Length ?? -1);                   // -1
```

Q225. Output: exceptions in finally.

```
int F() { try { return 1; } finally { Console.Write("finally "); } }
Console.Write(F());   // finally 1
```

Finally runs before the method returns; you can't `return` from finally in C#.

Q226. Output: return value modified in finally.

```
int G() { int x = 1; try { return x; } finally { x = 2; } }   // returns 1 (value already captured)
```

Q227. Output: exception thrown in finally.

If finally throws, the original exception is lost and the new one propagates. Avoid throwing from finally.

Q228. Output: static constructor timing.

```
class S { static S() { Console.Write("static "); } public static int V = 5; }
Console.Write("start "); Console.Write(S.V);   // start static 5
```

Q229. Output: static field shared across instances.

```
class C { public static int Count; public C() { Count++; } }
new C(); new C(); Console.Write(C.Count);   // 2
```

Q230. Output: generic static field per closed type.

```
class G<T> { public static int N; }
G<int>.N = 1; G<string>.N = 2; Console.Write(G<int>.N);   // 1 (each closed type has its own static)
```

Q231. Output: LINQ deferred execution with variable capture.

```
int min = 2; var q = new[] { 1, 2, 3, 4 }.Where(x => x > min); min = 3;
Console.Write(q.Count());   // 1 (uses min = 3 at enumeration)
```

Q232. Output: multiple enumeration side effect.

```
var q = Enumerable.Range(1, 3).Select(x => { Console.Write(x); return x; });
var c = q.Count(); var f = q.First();   // prints 1 2 3 then 1
```

Q233. Output: async method without await.

```
async Task<int> A() { Console.Write("A "); return 1; }   // warning: runs synchronously
var t = A(); Console.Write("B ");   // A B
```

Q234. Output: task exception observation.

```
var t = Task.Run(() => throw new Exception("x"));   // no crash until awaited/Result
try { await t; } catch (Exception e) { Console.Write(e.Message); }   // x
```

`await` rethrows the first exception; `.Wait()/.Result` wrap in AggregateException.

Q235. Output: Task.WhenAll with multiple failures.

`await Task.WhenAll(t1, t2)` throws the first exception; inspect `whenAllTask.Exception.InnerExceptions` for all.

Q236. Output: struct copy in foreach.

```
var ps = new[] { new P { X = 1 } };
foreach (var p in ps) { /* p.X = 2; compile error: iteration variable is readonly */ }
```

Q237. Output: enum default and casting.

```
enum Status { Pending = 1, Done = 2 }
Status s = default; Console.Write(s);          // 0 (not a defined name)
Console.Write((Status)5);                      // 5 (any int is allowed!)
Console.Write(Enum.IsDefined(typeof(Status), 5)); // False
```

Validate enum input from APIs.

Q238. Output: switch fall-through.

C# forbids implicit fall-through; each non-empty case must end with break/return/goto. Empty cases can stack: `case 1: case 2: ...`.

Q239. Output: `??=` and `?.` combination.

```
List<int>? l = null; (l ??= new()).Add(1); Console.Write(l.Count);   // 1
```

Q240. Output: string.Equals culture issue.

`"I".ToLower()` in Turkish culture yields "ı". Use `ToLowerInvariant()` or `string.Equals(a, b, StringComparison.OrdinalIgnoreCase)` for identifiers.

Q241. Output: DateTime vs DateTimeOffset.

`DateTime.Now` has Kind Local, no offset info when serialised across servers. Use `DateTimeOffset` or UTC (`DateTime.UtcNow`) for timestamps; convert for display.

Q242. Output: Dictionary key mutation.

If you mutate an object used as a Dictionary key so its hash changes, lookups fail (key "lost"). Keys should be immutable.

Q243. Output: Dictionary indexer vs Add.

`dict["k"] = v` adds or overwrites; `dict.Add("k", v)` throws ArgumentException if the key exists; `dict["missing"]` throws KeyNotFoundException; use `TryGetValue`.

Q244. Output: modifying a collection while iterating.

```
foreach (var x in list) if (x < 0) list.Remove(x);   // InvalidOperationException
list.RemoveAll(x => x < 0);                          // correct
```

Q245. Output: ToString on decimal formatting.

```
12.5m.ToString("F2")  // 12.50
1234567.891m.ToString("N2", new CultureInfo("en-IN"))  // 12,34,567.89
```

Q246. Output: params with null.

`void M(params string[] a) => Console.Write(a == null);` then `M(null)` prints True (null array), `M()` prints False (empty array).

Q247. Output: optional parameters and overriding.

Default values are taken from the static type at compile time:

```
class A { public virtual void M(int x = 1) => Console.Write(x); }
class B : A { public override void M(int x = 2) => Console.Write(x); }
A a = new B(); a.M();   // 1 (default from A, method from B)
```

Q248. Output: readonly struct field mutation via method.

Calling a mutating method on a `readonly` field of struct type operates on a defensive copy, so the field doesn't change. Mark structs `readonly` to avoid confusion.

Q249. Output: lock on value type or string.

`lock(5)` doesn't compile (value types boxed each time). Locking on a string literal or `this`/typeof is dangerous because others can lock the same object. Use a private readonly object.

Q250. Output: using with a null resource.

`using (Stream s = null) { }` is fine; Dispose isn't called on null.

Q251. Output: `var` with null.

`var x = null;` doesn't compile (no type to infer).

Q252. Output: char arithmetic.

```
Console.Write('a' + 1);        // 98
Console.Write((char)('a' + 1)); // b
Console.Write("a" + 1);        // a1
```

Q253. Output: string.Split edge cases.

```
"a,,b".Split(',').Length                                    // 3
"a,,b".Split(',', StringSplitOptions.RemoveEmptyEntries).Length  // 2
```

Q254. Output: Interlocked vs volatile.

`volatile` ensures visibility of the latest value but not atomic read-modify-write; `count++` on a volatile field is still a race. Use Interlocked.

Q255. Output: Lazy<T> evaluation.

`var l = new Lazy<int>(() => { Console.Write("init "); return 5; });` prints nothing until `l.Value` is accessed, and only once.

Q256. Output: event with no subscribers.

`MyEvent(this, e)` throws NullReferenceException if no subscribers; use `MyEvent?.Invoke(this, e)`.

Q257. Output: GetHashCode of strings across runs.

In .NET Core, `string.GetHashCode()` is randomised per process. Never persist it; use a stable hash (SHA-256, xxHash).

Q258. Output: boxing equality.

```
object a = 5, b = 5; Console.Write(a == b);        // False (two boxes)
Console.Write(a.Equals(b));                         // True
```

15. More C# Language Questions

Q259. What are the C# versions and key features you should know?

C# 7: tuples, pattern matching, local functions, out var. C# 8: nullable reference types, async streams, switch expressions, default interface methods, ranges/indices. C# 9: records, init, top-level statements, target-typed new. C# 10: global usings, file-scoped namespaces, record structs. C# 11: raw string literals, required members, list patterns, generic math. C# 12: primary constructors, collection expressions, alias any type. C# 13: params collections, new Lock type, `field` keyword preview. C# 14 (with .NET 10): extension members, `field` keyword, null-conditional assignment.

Q260. What are local functions vs lambdas?

Local functions are named methods inside a method; can be static (no captures), recursive, generic, and avoid delegate allocation. Lambdas are anonymous and create delegates.

Q261. What are ranges and indices?

`arr[^1]` last element, `arr[1..4]` slice from 1 to 3, `s[..3]` first three chars. Uses `System.Index` and `System.Range`.

Q262. What are raw string literals?

`"""` delimited strings with no escaping, good for JSON/SQL templates; indentation trimmed from the closing quotes.

Q263. What is the `required` modifier?

Forces object initializers to set the member: `public required string Rrn { get; init; }`.

Q264. What are top-level statements and global usings?

Program.cs without Main boilerplate; `global using` applies a namespace to the whole project (ImplicitUsings).

Q265. What is file-scoped namespace?

`namespace Recon.Core;` applies to the whole file, reducing indentation.

Q266. What is target-typed new?

`List<Txn> list = new();` when the type is known from context.

Q267. What is a discard?

`_` to ignore values: `_ = Task.Run(...)`, `var (_, count) = Stats();`, `_ => default` in switch.

Q268. What is the `default` literal and `default(T)`?

Zero/null value for a type: `default(int)` = 0, `default(string)` = null. In generics, `default` adapts to T.

Q269. What is the difference between `string` and `String`?

`string` is the C# alias for `System.String`; identical.

Q270. What is string interpolation vs String.Format vs concatenation performance?

Interpolation compiles to efficient `DefaultInterpolatedStringHandler` (.NET 6+). For loops use StringBuilder. For logging use message templates, not interpolation (avoids formatting when level disabled).

Q271. What is `StringComparison.Ordinal` and when to use it?

Byte-by-byte comparison: fastest and culture-independent; use for identifiers, keys, file paths. Use culture-aware comparison for user-facing sorting.

Q272. What is `IFormatProvider` / CultureInfo issue with parsing numbers?

`decimal.Parse("1,234.50")` fails in cultures using comma as decimal separator. Use `CultureInfo.InvariantCulture` when parsing files.

Q273. What are checked and unchecked contexts?

`checked` throws OverflowException on integral overflow; `unchecked` wraps silently (default). Decimal always throws on overflow.

Q274. What is the `dynamic` keyword's cost?

Runtime binder (DLR) resolves members at runtime, slower and no compile-time safety. Used for COM interop or dynamic JSON (prefer JsonNode/JsonElement).

Q275. What is `ExpandoObject`?

A dynamic object you can add members to at runtime.

Q276. What are expression trees?

Code represented as data (`Expression<Func<T, bool>>`). EF Core translates them to SQL. Can be compiled into delegates at runtime.

Q277. What is the difference between `Func<T,bool>` and `Expression<Func<T,bool>>` in EF?

A `Func` forces client-side evaluation (loads data then filters in memory), while `Expression` can be translated to SQL. Repository methods taking `Func` cause full table scans in memory.

Q278. What are source generators?

Compile-time code generation (Roslyn) used by System.Text.Json, logging (`LoggerMessage`), regex (`GeneratedRegex`), reducing reflection and enabling AOT.

Q279. What is `GeneratedRegex` and Regex performance?

`[GeneratedRegex("^[0-9]{12}$")] private static partial Regex RrnRegex();` generates optimised code at compile time. Otherwise cache Regex instances or use `RegexOptions.Compiled`; set timeouts to prevent ReDoS.

Q280. System.Text.Json vs Newtonsoft.Json.

STJ is built-in, faster, lower allocations, supports source generation and AOT, stricter defaults (case-sensitive, no comments). Newtonsoft has more features (polymorphism, flexible settings), still common in older code. .NET 7+ added polymorphic support to STJ.

Q281. How do you customise JSON serialization?

`JsonSerializerOptions` (camelCase, ignore nulls), attributes (`[JsonPropertyName]`, `[JsonIgnore]`, `[JsonConverter]`), custom converters for formats like dates or money.

Q282. What are attributes like [Obsolete], [Conditional], [CallerMemberName]?

`[Obsolete]` warns on use; `[Conditional("DEBUG")]` removes calls in release; caller info attributes inject caller name/file/line for logging.

Q283. What is the difference between `Assembly`, `AppDomain` and `AssemblyLoadContext`?

AppDomains (isolation units) are not supported in .NET Core beyond the default one. `AssemblyLoadContext` loads/unloads assemblies (plugin systems).

Q284. What is NuGet and central package management?

NuGet is the package manager. Central Package Management (`Directory.Packages.props`) keeps versions consistent across projects.

Q285. What is the difference between Debug and Release builds?

Release enables optimisations, removes debug-only code, smaller and faster; Debug includes symbols and no optimisations for stepping.

Q286. What is .NET Standard vs .NET Core vs .NET 5+?

.NET Standard is an API specification to share libraries across .NET Framework and Core. From .NET 5 onward, target `net8.0` etc.; use `netstandard2.0` only to support .NET Framework consumers.

Q287. What is Native AOT?

Compiles the app to native code ahead of time: fast startup, small memory, no JIT. Limitations: no runtime reflection emit, trimming requirements. Good for microservices and serverless.

Q288. What is trimming?

Removing unused code from published self-contained apps; reflection-heavy code needs annotations.

Q289. Framework-dependent vs self-contained deployment.

Framework-dependent requires the runtime installed (small). Self-contained bundles the runtime (bigger, no install needed). Single-file publish packs into one executable.

Q290. What is `IEnumerable<T>` covariance useful for in APIs?

You can pass `IEnumerable<CardTxn>` where `IEnumerable<Txn>` is expected without copying.

Q291. What is the difference between `throw` and `throw ex` in a catch? Output of stack trace.

`throw;` preserves the original stack trace. `throw ex;` resets it to the current line, losing where the error originated. Use `ExceptionDispatchInfo.Capture(ex).Throw()` to rethrow later with stack.

Q292. What is an inner exception?

Wrapping a low-level exception in a domain exception while preserving the cause: `throw new ReconException("Load failed", ex);`.

Q293. Is catching `Exception` always bad?

At boundaries (global handler, background job loop) it's appropriate, with logging. Inside business logic, catch specific exceptions you can handle.

Q294. What is `when` in catch (exception filters) advantage?

Filters don't unwind the stack if false, preserving debug context, and allow catching by condition.

Q295. Which exceptions should you never catch?

StackOverflowException (can't be caught), OutOfMemoryException (usually can't recover), AccessViolation. ThreadAbortException doesn't exist in .NET Core.

Q296. What is the difference between ArgumentException, ArgumentNullException, InvalidOperationException, NotSupportedException?

Argument*: bad input values. InvalidOperation: object's state doesn't allow the call (e.g. approving an already-approved batch). NotSupported: operation not supported by the type at all.

Q297. What is the Result pattern vs exceptions?

Return `Result<T>` with success/failure for expected business failures (validation, not found) instead of throwing; exceptions for unexpected errors. Improves performance and clarity.

Q298. What are memory leaks possible in managed code?

Static references, event handlers, timers not disposed, caches without limits, closures capturing large objects, unmanaged resources not disposed, pinned objects.

Q299. What is `GC.Collect()` and should you call it?

Forces a collection; almost never call it in production; it hurts performance by promoting objects and disrupting heuristics.

Q300. What is a weak reference?

`WeakReference<T>` holds an object without preventing GC; useful for caches.

Q301. What are `ArrayPool<T>` and `ObjectPool<T>`?

Pools reusable arrays/objects to reduce allocations in hot paths (e.g. buffers for reading files or network).

Q302. What is `stackalloc`?

Allocates memory on the stack, used with `Span<T>`: `Span<byte> buf = stackalloc byte[256];`. Small sizes only.

Q303. What is `ref struct`?

A struct that can only live on the stack (like Span): can't be boxed, captured by lambdas, or used across await.

Q304. What is `ref return` and `ref local`?

Return a reference to a storage location instead of a copy, used in high-performance code to modify array elements in place.

Q305. What is the `unsafe` keyword?

Allows pointers and manual memory operations; requires `AllowUnsafeBlocks`. Rarely needed.

Q306. What is `IntPtr`/`nint` and P/Invoke?

Native-sized integers and calling native DLL functions via `[DllImport]`/`[LibraryImport]` (e.g. HSM vendor libraries).

Q307. What is the difference between `Thread.Sleep` and `Task.Delay`?

Sleep blocks the thread; Delay creates a timer-based task and frees the thread while waiting. Use Task.Delay in async code.

Q308. What is the difference between `Convert.ToInt32`, `int.Parse` and `int.TryParse`?

Parse throws on null/invalid; Convert returns 0 for null and throws on invalid; TryParse returns bool without exceptions (best for input).

Q309. What does `sealed` give for performance?

JIT can devirtualise calls on sealed types; .NET analysers suggest sealing internal classes.

Q310. What are generic math interfaces (INumber<T>)?

C# 11 static abstract interface members allow writing `T Sum<T>(IEnumerable<T> xs) where T : INumber<T>` that works for int, decimal, double.

Q311. What is the `with` expression on structs and anonymous types?

Since C# 10 works on structs and anonymous types, not only records.

16. Collections and LINQ (More)

Q312. How does Dictionary work internally?

Array of buckets and entries; key's hash code modulo bucket count picks a bucket; collisions chained via entry indices; resizes to a prime size when full. Average O(1), worst O(n) with bad hashes.

Q313. What happens if two keys have the same hash code?

They collide; Equals decides if they're the same key. Correctness is maintained; performance degrades if many collisions.

Q314. How do you use a custom comparer in Dictionary/HashSet?

`new Dictionary<string, Txn>(StringComparer.OrdinalIgnoreCase)` or a custom `IEqualityComparer<T>` (e.g. composite key comparer).

Q315. What is a good composite key for a Dictionary?

A tuple `(string Rrn, decimal Amount)` or record struct; both have value equality and good hash codes.

Q316. List capacity vs Count.

Count is the number of items; Capacity is allocated size. Pre-size with `new List<T>(expected)` to avoid repeated resizing when loading big files.

Q317. ReadOnlyCollection vs ImmutableList vs FrozenDictionary.

ReadOnlyCollection is a read-only wrapper (underlying list can still change). Immutable collections never change (new copy on modification, structural sharing). FrozenDictionary/FrozenSet (.NET 8) are optimised for read-heavy, never-changing lookups (e.g. BIN tables, config codes).

Q318. SortedList vs SortedDictionary.

SortedList uses sorted arrays (less memory, O(n) inserts, index access). SortedDictionary is a red-black tree (O(log n) inserts). Use SortedDictionary for frequent inserts.

Q319. What is a BitArray?

Compact array of bits; useful for flags or bitmap parsing (ISO 8583 bitmap).

Q320. What does yield break do?

Ends an iterator early.

Q321. What is the difference between `IEnumerable.Count()` and `List.Count`?

`Count` property is O(1). `Count()` extension checks for ICollection and uses Count if possible, otherwise enumerates.

Q322. What is `ToLookup` vs `GroupBy`?

`ToLookup` executes immediately and creates an indexed, immutable multi-map; GroupBy is deferred.

Q323. What is the difference between `Where().FirstOrDefault()` and `FirstOrDefault(predicate)`?

Same result; both lazy. In EF both translate to TOP 1. Minor style difference.

Q324. How do you do pagination with LINQ?

`.OrderBy(x => x.Id).Skip((page - 1) * size).Take(size)`; always order before paging. .NET 6: `Chunk(size)` for batching in memory.

Q325. What are Zip, Aggregate, Distinct, DistinctBy, MaxBy?

`Zip` pairs two sequences; `Aggregate` folds; `DistinctBy(x => x.Rrn)` (.NET 6) distinct by key; `MaxBy(x => x.Amount)` returns the element with max key.

Q326. Write LINQ: find employees with salary greater than department average.

```
var res = emps.GroupBy(e => e.Dept).SelectMany(g => g.Where(e => e.Salary > g.Average(x => x.Salary)));
```

Q327. Write LINQ: nth highest salary.

```
var nth = emps.Select(e => e.Salary).Distinct().OrderByDescending(s => s).ElementAtOrDefault(n - 1);
```

Q328. Write LINQ: count characters in a string.

```
var counts = s.GroupBy(c => c).Select(g => new { Char = g.Key, Count = g.Count() });
```

Q329. Write LINQ: find pairs that sum to target.

```
var pairs = from a in nums from b in nums where a < b && a + b == target select (a, b);
```

Q330. Write LINQ: pivot amounts by month.

```
var pivot = txns.GroupBy(t => t.Merchant).Select(g => new { g.Key,
   Jan = g.Where(t => t.Date.Month == 1).Sum(t => t.Amount), Feb = g.Where(t => t.Date.Month == 2).Sum(t
    => t.Amount) });
```

Q331. Write LINQ: running total.

```
decimal run = 0; var withRunning = txns.OrderBy(t => t.Date).Select(t => new { t.Date, t.Amount, Running
    = run += t.Amount });
```

(Side effect in Select; fine in memory but not translatable to SQL; in SQL use window function `SUM() OVER`.)

Q332. Write LINQ: full outer join.

Union of a left join and a right anti-join; or build HashSets of keys and union.

Q333. What is query syntax vs method syntax?

Query syntax (`from x in ... where ... select`) compiles to method calls. Some operators (Count, Distinct) only exist as methods. Joins are often more readable in query syntax.

Q334. What is `let` in query syntax?

Introduces a range variable for intermediate computation: `let total = o.Lines.Sum(l => l.Amount)`.

Q335. What is PLINQ?

Parallel LINQ: `.AsParallel()` for CPU-bound queries on large data; order not preserved unless `AsOrdered()`. Not for I/O or small data.

Q336. IQueryable gotcha: calling a C# method inside Where.

EF Core can't translate custom methods and throws (or evaluates client-side only in the final Select). Rewrite using translatable expressions or `EF.Functions` (e.g. `EF.Functions.Like`).

17. Async, Threading and Concurrency (More)

Q337. What is the ThreadPool and thread-pool starvation?

Shared pool of worker threads. Starvation happens when threads are blocked (sync-over-async, long blocking I/O), so new work waits; pool injects threads slowly. Symptoms: high latency, low CPU. Fix: async all the way.

Q338. What is SynchronizationContext?

An abstraction for where continuations run (UI thread in WPF/WinForms). ASP.NET Core has none, so continuations run on any pool thread.

Q339. What is TaskCompletionSource?

Creates a Task you complete manually; used to wrap callback/event-based APIs into async.

```
var tcs = new TaskCompletionSource<string>(TaskCreationOptions.RunContinuationsAsynchronously);
client.OnResponse += r => tcs.TrySetResult(r); return await tcs.Task;
```

Q340. Task.Run vs Task.Factory.StartNew.

`Task.Run` is the safe default (unwraps async lambdas, uses default scheduler). StartNew has more options but pitfalls (returns `Task<Task>` with async lambdas). Use `LongRunning` only for dedicated threads.

Q341. Should you use Task.Run in ASP.NET Core controllers?

No for I/O work; it just moves work to another pool thread with overhead. Use it only to offload CPU-bound work in UI apps, or fire background work properly via queue/BackgroundService.

Q342. Why is fire-and-forget dangerous in ASP.NET Core?

Unobserved exceptions, scoped services (DbContext) disposed after the request ends, and lost work on shutdown. Use a background queue (Channel + BackgroundService) or Hangfire.

Q343. What is async over sync and sync over async?

Sync over async: blocking on async (`.Result`) - deadlocks/starvation. Async over sync: wrapping sync code in Task.Run to look async - wastes threads in servers.

Q344. How do you run async code in a constructor?

You can't await in constructors. Use a static async factory (`CreateAsync`), lazy initialisation, or `IHostedService` startup tasks.

Q345. How do you implement async lazy initialisation?

`Lazy<Task<T>>` or `AsyncLazy<T>` so the expensive async init runs once.

Q346. What is a deadlock in multithreading and how to avoid?

Two threads each hold a lock the other needs. Avoid by lock ordering, timeouts (`Monitor.TryEnter`), minimal lock scope, and avoiding calling external code while holding locks.

Q347. What is a livelock and starvation?

Livelock: threads keep reacting to each other without progress. Starvation: a thread never gets resources due to others hogging them.

Q348. What is the difference between concurrency and parallelism?

Concurrency: dealing with many tasks at once (interleaving, async I/O). Parallelism: doing many computations at the same time on multiple cores.

Q349. What are thread-safe collections in .NET?

ConcurrentDictionary, ConcurrentQueue, ConcurrentStack, ConcurrentBag, BlockingCollection, Channels, Immutable collections.

Q350. What is ThreadLocal and AsyncLocal?

ThreadLocal stores per-thread values. AsyncLocal flows a value along the async execution context (used for correlation IDs, Activity, logging scopes).

Q351. What is a Timer in .NET? Which one to use?

System.Threading.Timer (pool callbacks), System.Timers.Timer (event-based), PeriodicTimer (.NET 6, async-friendly loop, no overlapping ticks). Use PeriodicTimer in BackgroundService.

Q352. What is the volatile keyword?

Prevents certain compiler/CPU reorderings and caching for a field, ensuring reads see the latest write. Not a replacement for locks.

Q353. What is the Monitor.Wait/Pulse?

Low-level signalling inside a lock for producer-consumer; generally replaced by higher-level primitives (SemaphoreSlim, Channels).

Q354. What are ManualResetEventSlim and CountdownEvent?

Signalling primitives: MRE lets threads wait until set; CountdownEvent waits until a count reaches zero.

Q355. What is Parallel.For with thread-local state?

Overloads with localInit/localFinally to accumulate per-thread totals then merge, avoiding contention on a shared variable.

Q356. How do you handle exceptions in Parallel.ForEach?

They're collected into an AggregateException thrown at the end; use `ParallelLoopState.Stop()` to stop early.

Q357. How do you implement timeouts for async operations?

`CancellationTokenSource(TimeSpan)` or `.WaitAsync(TimeSpan, ct)` (.NET 6), and HttpClient.Timeout.

Q358. What is `IAsyncDisposable` and when to use?

For resources that need async cleanup (flushing streams, closing DB connections). Use `await using`.

Q359. How would you process 1 million records with limited memory and good throughput?

Stream input (IAsyncEnumerable), bounded Channel between parser and N consumers, batch inserts with SqlBulkCopy, CancellationToken, progress reporting, and backpressure.

Q360. How do you report progress from an async method?

`IProgress<T>` with `Progress<T>` callback: `progress?.Report(percent)`.

Q361. Is `async` always faster?

No. It improves scalability (threads freed during I/O), not raw speed; it adds state machine overhead. CPU-bound work doesn't benefit.

Q362. What is `ConfigureAwait(ConfigureAwaitOptions)` in .NET 8?

New options such as `SuppressThrowing` and `ForceYielding` for advanced scenarios.

18. ASP.NET Core (More In-Depth)

Q363. What is the request lifecycle in ASP.NET Core?

Kestrel receives the request, creates HttpContext, it flows through middleware (exception handling, HTTPS, static files, routing, CORS, auth, authorization), endpoint is selected, filters run (MVC), model binding and validation, action executes, result is executed, response flows back through middleware.

Q364. What is HttpContext and how do you access it in services?

Per-request object with Request, Response, User, Items, RequestServices. In services inject `IHttpContextAccessor` (register with `AddHttpContextAccessor`); avoid capturing it in singletons beyond the request.

Q365. What is endpoint routing?

Routing decouples endpoint selection (UseRouting) from execution (endpoints), letting middleware in between (auth, CORS) know the selected endpoint and its metadata.

Q366. How does ASP.NET Core choose between two matching routes?

By route precedence: literal segments beat parameters, constrained parameters beat unconstrained, then order. Ambiguous matches throw AmbiguousMatchException.

Q367. What is short-circuiting middleware?

Middleware that doesn't call `next` (returns response early), e.g. static files, health checks, `ShortCircuit()` on endpoints in .NET 8.

Q368. How do you write conventional middleware vs IMiddleware?

Conventional: class with `InvokeAsync(HttpContext)` and constructor `RequestDelegate next`, instantiated once (singleton-like; inject scoped services in InvokeAsync parameters). `IMiddleware`: resolved from DI per request, supports scoped lifetime.

Q369. Why can't you inject a scoped service into conventional middleware's constructor?

Middleware is constructed once at startup; scoped services must be injected into `InvokeAsync` parameters instead.

Q370. What are action filters? Show one.

```
public class ValidateModelFilter : IActionFilter {
  public void OnActionExecuting(ActionExecutingContext ctx) { if (!ctx.ModelState.IsValid) ctx.Result =
    new BadRequestObjectResult(ctx.ModelState); }
  public void OnActionExecuted(ActionExecutedContext ctx) { } }
builder.Services.AddControllers(o => o.Filters.Add<ValidateModelFilter>());
```

Q371. Types of filters and their order?

Authorization, Resource, Action, Exception, Result (plus Endpoint filters for minimal APIs). Global, controller and action scope ordering via `Order` property.

Q372. What is ServiceFilter vs TypeFilter?

`[ServiceFilter(typeof(AuditFilter))]` resolves the filter from DI (must be registered). `[TypeFilter]` creates it with DI for constructor args without registration.

Q373. How do you log every request/response body?

Middleware enabling buffering (`Request.EnableBuffering()`), reading and resetting the stream, wrapping Response.Body with a MemoryStream. Or built-in `AddHttpLogging`. Mask sensitive fields and limit body size.

Q374. How do you return files from an API?

`return File(bytes, "application/pdf", "report.pdf");` or `PhysicalFile`, or stream with `File(stream, contentType)` for large files.

Q375. How do you upload large files?

Stream using `MultipartReader` or `IFormFile` with increased limits (`FormOptions.MultipartBodyLengthLimit`, Kestrel `MaxRequestBodySize`), validate content type/extension/signature, save outside wwwroot, scan for malware.

Q376. What is IFormFile and its limitations?

Abstraction for uploaded file; buffered to memory/disk; fine for small/medium files. For very large files use streaming.

Q377. How do you implement pagination in an API response?

Query params `page`, `pageSize` (with max), return items plus metadata `{ total, page, pageSize }` or Link headers. Prefer keyset pagination for large tables.

Q378. How do you implement filtering and sorting safely?

Whitelist sortable columns (map strings to expressions), validate filters, avoid dynamic SQL concatenation.

Q379. What is content negotiation?

Server selects response format (JSON, XML) based on `Accept` header; enable XML with `AddXmlSerializerFormatters`.

Q380. What is ProblemDetails?

RFC 7807/9457 standard error format: `type, title, status, detail, instance` plus extensions. ASP.NET Core supports it via `AddProblemDetails`.

Q381. What is output caching vs response caching?

Response caching relies on HTTP cache headers and client/proxy caching. Output caching (.NET 7) caches responses on the server with policies, tag invalidation, and works regardless of client headers.

Q382. How do you implement ETags?

Compute a version/hash of the resource; return `ETag`; client sends `If-None-Match` (return 304) or `If-Match` for updates (412 if changed) for optimistic concurrency.

Q383. What is response compression?

`AddResponseCompression` with Brotli/Gzip for text payloads; careful with HTTPS and secrets (BREACH attack).

Q384. How do you implement background job queue in ASP.NET Core?

```
public class JobQueue { private readonly Channel<Func<IServiceProvider, CancellationToken, Task>> ch =
    Channel.CreateUnbounded<...>();
  public ValueTask EnqueueAsync(Func<IServiceProvider, CancellationToken, Task> job) => ch.Writer.
    WriteAsync(job);
  public IAsyncEnumerable<...> ReadAllAsync(CancellationToken ct) => ch.Reader.ReadAllAsync(ct); }
```

A BackgroundService reads jobs and executes them in a new DI scope.

Q385. IHostedService vs BackgroundService.

IHostedService has StartAsync/StopAsync. BackgroundService is an abstract base implementing it with `ExecuteAsync` for long-running loops.

Q386. What is a Worker Service?

A .NET project template for long-running background processes (Windows Service, systemd, containers) using the Generic Host with DI, config and logging.

Q387. How do you run a .NET app as a Windows Service?

`builder.Services.AddWindowsService()` (Microsoft.Extensions.Hosting.WindowsServices), publish, then `sc create`. For Linux use `AddSystemd()`.

Q388. What is the Generic Host?

`Host.CreateApplicationBuilder` providing DI, configuration, logging and lifetime for any app type (web, worker, console).

Q389. How do you gracefully shut down?

Respect CancellationToken in hosted services, `IHostApplicationLifetime.ApplicationStopping` for cleanup, configure `HostOptions.ShutdownTimeout`.

Q390. How do you read configuration in a class library?

Inject `IOptions<T>` or `IConfiguration`; don't read appsettings directly.

Q391. How do you reload configuration without restart?

`reloadOnChange: true` for JSON files and `IOptionsMonitor<T>` / `IOptionsSnapshot<T>` to pick up changes.

Q392. How do you validate options at startup?

`.ValidateDataAnnotations().ValidateOnStart()` or `IValidateOptions<T>` for custom logic; app fails fast with clear errors.

Q393. What is the difference between AddSingleton(instance) and AddSingleton<T>()?

Instance registration uses your object (container won't dispose it). Type registration lets the container create and dispose it.

Q394. What is TryAdd and how do multiple registrations of the same interface behave?

Multiple Add calls: injecting `IService` gets the last one; `IEnumerable<IService>` gets all. `TryAdd*` registers only if none exists (library defaults).

Q395. How do you resolve services manually and why is service locator an anti-pattern?

`provider.GetRequiredService<T>()`. Service locator hides dependencies and makes testing harder; use constructor injection; manual resolution only in factories/infrastructure.

Q396. How do you register open generics?

`services.AddScoped(typeof(IRepository<>), typeof(Repository<>));`

Q397. How do you decorate a service in Microsoft DI?

No built-in support; use Scrutor (`services.Decorate<IRepo, CachedRepo>()`) or manual factory registration.

Q398. What happens if a constructor has a dependency not registered?

Runtime exception when resolving ("Unable to resolve service for type"). `ValidateOnBuild` catches it at startup.

Q399. What is the difference between `ILogger<T>` and `ILoggerFactory`?

`ILogger<T>` uses the type name as category. `ILoggerFactory.CreateLogger("name")` for custom categories.

Q400. What is high-performance logging with LoggerMessage?

Source-generated logging methods avoid boxing and parsing templates:

```
[LoggerMessage(Level = LogLevel.Information, Message = "Batch {BatchId} matched {Count}")]
static partial void LogMatched(ILogger logger, long batchId, int count);
```

Q401. What is Serilog and structured logging sinks?

Serilog writes structured events to sinks (Console, File, Seq, Elasticsearch, SQL). Configure enrichers (machine, correlation ID) and minimum levels per namespace.

Q402. How do you implement multi-tenancy?

Resolve tenant from host/header/claim via middleware, store in a scoped `ITenantContext`, apply EF global query filters by TenantId or separate connection strings per tenant.

Q403. How do you implement localization?

`AddLocalization`, resource files (.resx), `IStringLocalizer<T>`, RequestLocalization middleware with supported cultures.

Q404. What is SignalR's scale-out problem?

Connections are tied to a server; messages to groups need a backplane (Redis) or Azure SignalR Service so all servers receive them.

Q405. What is gRPC-Web and why?

Browsers can't use raw gRPC (HTTP/2 trailers); gRPC-Web adds a compatible protocol via middleware.

Q406. How do you secure Swagger in production?

Disable it or protect with authentication; avoid exposing internal endpoints.

Q407. What is YARP?

Yet Another Reverse Proxy: Microsoft's library to build gateways/reverse proxies in ASP.NET Core with routing, load balancing and transforms.

Q408. What is .NET Aspire?

An opinionated stack for building observable, distributed apps: AppHost orchestration, service defaults (OpenTelemetry, health checks, resilience), integrations, and a developer dashboard.

Q409. What is minimal API endpoint filter?

```
app.MapPost("/pay", ...).AddEndpointFilter(async (ctx, next) => { /* validate */
return await next(ctx); });
```

Q410. How do you bind complex query parameters in minimal APIs?

`[AsParameters] SearchRequest req` binds properties from query/route/header.

Q411. What is anti-forgery in minimal APIs (.NET 8)?

`app.UseAntiforgery()` and form endpoints validate tokens automatically.

19. Entity Framework Core (More)

Q412. How does EF Core track changes? Entity states.

Detached, Unchanged, Added, Modified, Deleted. SaveChanges detects changes via snapshot comparison (DetectChanges) and generates SQL accordingly.

Q413. Attach vs Update vs Add.

Add: Added (INSERT). Attach: Unchanged (track existing without update). Update: marks all properties Modified (UPDATE all columns). For disconnected entities prefer loading and setting values or `Entry(e).Property(x).IsModified = true`.

Q414. What is the Fluent API vs Data Annotations?

Both configure the model; Fluent API (`OnModelCreating` or `IEntityTypeConfiguration<T>`) is more powerful and keeps entities clean.

Q415. How do you configure decimal precision?

`[Precision(18, 2)]` or `builder.Property(x => x.Amount).HasPrecision(18, 2);` otherwise EF warns and defaults may truncate.

Q416. How do you configure relationships?

```
b.HasMany(x => x.Txns).WithOne(t => t.Batch).HasForeignKey(t => t.BatchId).OnDelete(DeleteBehavior.
    Restrict);
```

Q417. What are owned types?

Value objects mapped into the owner's table (`OwnsOne(x => x.Address)`), no separate identity.

Q418. What are shadow properties?

Properties in the EF model but not on the CLR class (e.g. `CreatedAt`, `LastModifiedBy`), accessed via `EF.Property<DateTime>(e, "CreatedAt")`.

Q419. How do you implement audit fields automatically?

Override SaveChangesAsync or use a SaveChanges interceptor to set CreatedBy/ModifiedOn for Added/Modified entries.

Q420. What are interceptors in EF Core?

Hooks for commands, connections, transactions and SaveChanges: logging slow queries, adding query hints, setting session context.

Q421. How do you see generated SQL?

`ToQueryString()`, logging `LogTo(Console.WriteLine)`, `EnableSensitiveDataLogging()` (dev only), or SQL Profiler/Extended Events.

Q422. What is client vs server evaluation in EF Core 3+?

EF Core 3+ throws if a query part can't be translated (except final projection), preventing accidental in-memory filtering.

Q423. How do you handle raw SQL with EF for non-entity types?

`db.Database.SqlQuery<ReconSummary>($"SELECT ...")` (EF 8) for unmapped types.

Q424. What is TPH, TPT, TPC inheritance mapping?

Table-per-hierarchy (one table, discriminator; fastest), table-per-type (table per class; joins), table-per-concrete-type (EF 7; table per concrete class).

Q425. How do you seed data?

`HasData` in model (migrations) for static reference data, or seeding code at startup for environment-specific data.

Q426. How do you handle migrations in a team?

One migration per feature, rebase and regenerate on conflicts, review generated SQL, never edit applied migrations, idempotent scripts for deployment.

Q427. How do you manage connection resiliency?

`UseSqlServer(cs, o => o.EnableRetryOnFailure())` retries transient failures; with user transactions use the execution strategy (`db.Database.CreateExecutionStrategy().ExecuteAsync(...)`).

Q428. What is a keyless entity?

Entity without a primary key, mapped to views or raw SQL results (`HasNoKey()`); read-only.

Q429. How do you map stored procedures for CUD operations?

EF 7+: `InsertUsingStoredProcedure`, `UpdateUsingStoredProcedure`, `DeleteUsingStoredProcedure` in Fluent API.

Q430. How do you use temporal tables with EF?

`ToTable(t => t.IsTemporal())` and query history with `TemporalAsOf(date)`; good for audit requirements.

Q431. What is IDbContextFactory used for?

Creating short-lived DbContexts in Blazor, background services or parallel work where scoped lifetime doesn't fit.

Q432. What is the difference between Find and FirstOrDefault?

`Find` checks the change tracker first (no DB call if tracked) and only works with primary key. FirstOrDefault always queries.

Q433. How do you do a bulk delete in EF Core 7+?

```
await db.Logs.Where(l => l.Date < cutoff).ExecuteDeleteAsync();
```

Q434. Why is `Include` with filter useful?

Filtered include: `.Include(b => b.Txns.Where(t => t.Status == "UNMATCHED"))` loads only relevant children.

20. SQL Server Questions for .NET Developers

Q435. Clustered vs non-clustered index.

Clustered: defines physical order of rows (one per table, usually the PK). Non-clustered: separate B-tree with key and row locator; many allowed. Covering indexes include needed columns (`INCLUDE`) to avoid key lookups.

Q436. What is a key lookup and how to fix it?

Non-clustered index seek followed by fetching missing columns from the clustered index per row. Fix with INCLUDE columns or a better index.

Q437. Index seek vs scan.

Seek navigates the B-tree to specific rows (efficient). Scan reads all pages. Non-SARGable predicates cause scans: `WHERE YEAR(TxnDate) = 2026`, functions on columns, leading wildcard `LIKE '%x'`, implicit conversions.

Q438. What is SARGability? Rewrite example.

`WHERE CONVERT(date, TxnDate) = @d` -> `WHERE TxnDate >= @d AND TxnDate < DATEADD(day, 1, @d)`.

Q439. What is parameter sniffing?

SQL Server compiles a plan using the first parameter values; a plan optimal for a small bank may be terrible for a large one. Fixes: `OPTION (RECOMPILE)`, `OPTIMIZE FOR`, local variables, plan guides, Query Store forcing.

Q440. What are isolation levels?

Read Uncommitted (dirty reads, NOLOCK), Read Committed (default), Repeatable Read, Serializable, Snapshot, and Read Committed Snapshot (RCSI: readers don't block writers using row versioning).

Q441. Why is NOLOCK dangerous?

Dirty reads, missing or duplicate rows during page splits. Unacceptable for financial reconciliation; prefer RCSI.

Q442. What is a deadlock and how does SQL Server resolve it?

Two sessions waiting on each other's locks; SQL Server kills one (victim, error 1205). Analyse deadlock graphs, consistent access order, indexes, shorter transactions, retry.

Q443. Temp table vs table variable vs CTE.

Temp tables (#t): statistics, indexes, good for large intermediate data. Table variables (@t): limited statistics (improved in 2019 with deferred compilation), good for small sets. CTE: named query expression, not materialised, recursive support.

Q444. What are window functions? Examples.

```
SELECT Rrn, Amount, ROW_NUMBER() OVER (PARTITION BY Rrn ORDER BY CreatedAt DESC) AS rn,
       SUM(Amount) OVER (PARTITION BY MerchantId ORDER BY TxnDate ROWS UNBOUNDED PRECEDING) AS
    RunningTotal
FROM dbo.Txn;
```

Also RANK, DENSE_RANK, LAG/LEAD, NTILE.

Q445. Delete duplicates keeping the latest.

```
WITH d AS (SELECT *, ROW_NUMBER() OVER (PARTITION BY Rrn, Stan ORDER BY Id DESC) rn FROM dbo.Txn)
DELETE FROM d WHERE rn > 1;
```

Q446. Find the second highest salary in SQL.

`SELECT MAX(Salary) FROM Emp WHERE Salary < (SELECT MAX(Salary) FROM Emp);` or `DENSE_RANK() = 2`.

Q447. INNER vs LEFT vs FULL vs CROSS JOIN vs APPLY.

Inner: matching rows. Left: all left plus matches. Full: all from both. Cross: Cartesian product. CROSS/OUTER APPLY: run a table-valued expression per row (e.g. top 1 per group).

Q448. WHERE vs HAVING.

WHERE filters rows before grouping; HAVING filters groups after aggregation.

Q449. UNION vs UNION ALL.

UNION removes duplicates (sort/hash cost). UNION ALL keeps all and is faster.

Q450. DELETE vs TRUNCATE vs DROP.

DELETE: row by row, logged, WHERE allowed, fires triggers. TRUNCATE: deallocates pages, minimal logging, resets identity, no WHERE. DROP: removes the table.

Q451. Stored procedure vs function.

Procedures can modify data, return multiple result sets, use transactions and TRY/CATCH. Functions return a value/table, are usable in SELECT, can't modify data. Scalar UDFs can kill performance (inlining in 2019 helps).

Q452. What is MERGE and its caution?

Upsert in one statement (WHEN MATCHED / NOT MATCHED). Has known edge-case bugs and concurrency issues; use HOLDLOCK or separate UPDATE + INSERT patterns.

Q453. How do you handle errors and transactions in a stored procedure?

```
BEGIN TRY BEGIN TRAN; ... COMMIT; END TRY
BEGIN CATCH IF @@TRANCOUNT > 0 ROLLBACK; THROW; END CATCH
```

`SET XACT_ABORT ON` ensures rollback on errors.

Q454. What are table-valued parameters?

Pass a set of rows to a procedure as a user-defined table type; from .NET use a DataTable or `IEnumerable<SqlDataRecord>`. Good for batch operations.

Q455. How do you read an execution plan?

Look at the costliest operators, scans vs seeks, key lookups, estimated vs actual rows (bad statistics), spills (sort/hash warnings), implicit conversions, parallelism.

Q456. What are statistics?

Histograms of data distribution used by the optimizer for estimates. Outdated statistics cause bad plans; update them after big loads.

Q457. What is partitioning?

Splitting a large table by a key (e.g. TxnDate) into partitions; enables partition elimination and fast archiving via partition switching.

Q458. What is a computed column / indexed view?

Computed column derives a value (can be persisted and indexed). Indexed view materialises an aggregated view (strict requirements).

Q459. What is SQL injection prevention in stored procedures with dynamic SQL?

Use `sp_executesql` with parameters and `QUOTENAME` for object names; never concatenate user input.

Q460. What is `SET NOCOUNT ON`?

Stops "rows affected" messages, reducing network chatter; standard in procedures.

Q461. How do you find slow queries?

Query Store, `sys.dm_exec_query_stats`, Extended Events, Activity Monitor, and app-side telemetry.

21. Architecture, Microservices and DevOps (More)

Q462. What is Domain-Driven Design in short?

Model software around the business domain: ubiquitous language, bounded contexts, entities, value objects, aggregates (consistency boundaries), domain events, repositories.

Q463. What is an aggregate root?

Entity that controls access to a cluster of objects and enforces invariants; external code references only the root (e.g. `Batch` controls its `Txns`).

Q464. What are domain events vs integration events?

Domain events happen inside a bounded context (in-process, same transaction). Integration events are published to other services via a broker (after commit, via outbox).

Q465. What is eventual consistency?

Data across services becomes consistent over time rather than instantly; UI must handle intermediate states.

Q466. What is event sourcing?

Storing state as a sequence of events instead of current state; rebuild state by replaying. Great audit trail; complexity in querying (projections).

Q467. What is the API Gateway and BFF pattern?

Gateway: single entry point for routing, auth, rate limiting. Backend-for-Frontend: a gateway tailored for a specific client (web vs mobile).

Q468. What is service discovery?

Services find each other's addresses dynamically (Consul, Kubernetes DNS) instead of hardcoding.

Q469. What is idempotent message consumption?

Store processed message IDs (inbox table) and skip duplicates, because brokers deliver at least once.

Q470. RabbitMQ vs Kafka vs Azure Service Bus.

RabbitMQ: message broker with flexible routing, per-message ack. Kafka: distributed log, high throughput, replay, ordering per partition. Service Bus: managed broker with queues/topics, sessions, dead-lettering.

Q471. What is a dead-letter queue?

Where messages that repeatedly fail or expire go for inspection and reprocessing.

Q472. What is MassTransit / NServiceBus?

.NET messaging frameworks over brokers providing consumers, retries, sagas, outbox.

Q473. What is Kubernetes in brief?

Container orchestrator: deployments, pods, services, ingress, config maps, secrets, autoscaling, rolling updates, health probes.

Q474. What is CI/CD and how do you set it up for .NET?

CI: build, test, analyse on every push (GitHub Actions, Azure DevOps). CD: deploy artefacts/containers to environments with approvals. Steps: restore, build, test, publish, docker build/push, deploy, run migrations scripts.

Q475. What is the 12-factor app?

Principles for cloud apps: config in environment, stateless processes, backing services as resources, logs as event streams, dev/prod parity, disposability.

Q476. What is horizontal vs vertical scaling?

Vertical: bigger machine. Horizontal: more instances behind a load balancer (requires stateless apps, shared caches/sessions).

Q477. How do you handle session state in a load-balanced ASP.NET Core app?

Avoid server sessions; use JWT or distributed session store (Redis) and shared Data Protection keys.

Q478. What is observability (logs, metrics, traces)?

Logs: events; Metrics: numeric time series (request rate, latency, errors); Traces: request flow across services. OpenTelemetry standardises all three.

Q479. What is the strangler fig pattern?

Incrementally replacing a legacy system by routing features one by one to a new system until the old one can be removed.

Q480. What are feature flags in .NET?

`Microsoft.FeatureManagement` with `IFeatureManager.IsEnabledAsync("NewRecon")`, `[FeatureGate]`, and filters (percentage, time window, targeting).

22. Scenario and Rapid-Fire Round

Q481. A third-party SFTP drop must be processed exactly once. How do you design it?

Worker polls SFTP, records file name + checksum in a FileRegistry table with unique constraint before processing, moves file to processing/archive folders, processes in a transaction with status tracking, alerts on failures, supports manual reprocess.

Q482. An endpoint returns 500 only in production. How do you investigate?

Check logs with correlation ID, configuration differences (connection strings, missing secrets), environment-specific data, permissions, recent deployments; reproduce with prod-like data in staging.

Q483. Your API must call an SP that takes 2 minutes. How do you design it?

Make it async: return 202 with job ID, run the SP in a background job (increase command timeout), store status, notify via SignalR/poll; optimise the SP separately.

Q484. How do you version a database alongside code?

Migrations (EF or DbUp/Flyway scripts) stored in source control, applied in CI/CD, backward-compatible changes for zero downtime.

Q485. How do you protect configuration secrets in appsettings for on-prem deployments?

Environment variables, Windows DPAPI/Data Protection, encrypted sections, or a vault (HashiCorp Vault, Azure Key Vault); never plaintext in source control.

Q486. Value of `nameof(Txn.Amount)`?

"Amount".

Q487. What is the default access modifier for class members and top-level classes?

Members: private. Top-level types: internal.

Q488. Can a static class implement an interface?

No.

Q489. Can you declare a variable of an abstract class type?

Yes, and assign any concrete derived instance.

Q490. Can an interface inherit another interface?

Yes, multiple interfaces.

Q491. Can you have a static method in an interface?

Yes, since C# 8 (and static abstract since C# 11).

Q492. What is the base class of all types in .NET?

System.Object.

Q493. What does `sizeof(decimal)` return?

16 bytes.

Q494. What is the max value of int and long?

int: 2,147,483,647; long: 9,223,372,036,854,775,807.

Q495. What is the default value of a bool and a reference type field?

false and null.

Q496. Can `const` be used with reference types?

Only string (and null for other reference types).

Q497. Can you use `this` in a static method?

No.

Q498. Can we have multiple Main methods?

Yes, but you must specify the entry point with `<StartupObject>`.

Q499. Is string a value type?

No, reference type with value semantics for equality and immutability.

Q500. What is `Environment.NewLine` vs "\n"?

NewLine is platform-specific ("\r\n" on Windows, "\n" on Linux).

Q501. What is the difference between `Array.Copy` and `Clone`?

Both shallow; Clone creates a new array, Copy copies into an existing one with ranges.

Q502. What does `GC.SuppressFinalize` do?

Tells GC not to call the finalizer since Dispose already cleaned up.

Q503. Which HTTP verb for idempotent upsert?

PUT.

Q504. What is the use of `[FromServices]`?

Injects a service directly into an action method parameter.

Q505. What does `app.UseHsts()` do?

Adds the Strict-Transport-Security header so browsers only use HTTPS for the domain.

Q506. Which is faster: `Any()` or `Count() > 0` on List?

Both effectively O(1) for List (Any checks Count); for general IEnumerable and EF, Any is better.

Q507. What is the difference between `AddMvc`, `AddControllersWithViews` and `AddControllers`?

AddControllers: API only. AddControllersWithViews: MVC with views. AddMvc: everything including Razor Pages.

Q508. What is Razor Pages vs MVC vs Blazor?

Razor Pages: page-focused server rendering. MVC: controllers/views. Blazor: C# UI components running on server (SignalR) or WebAssembly; .NET 8 adds unified render modes.

Q509. What does `dotnet watch` do?

Rebuilds/hot reloads the app on file changes during development.